

Generative AI — Interview and Project Guide

This guide covers the foundations and modern systems behind generative AI. Every topic is self-contained and follows the same placement-oriented structure: intuition, working process, mathematics, implementation, evaluation, limitations, interview questions, and project practice.

Language Models

1. Overview

A language model (LM) assigns probabilities to sequences of tokens. Modern autoregressive LMs estimate the probability of the next token given previous tokens, while masked language models recover hidden tokens from surrounding context. Language models power autocomplete, search, translation, summarization, code assistants, chat systems, information extraction, and reasoning agents.

2. Intuition

An LM is a very advanced autocomplete system. After seeing **The capital of France is**, it assigns much more probability to **Paris** than to **banana**. It learns this behavior from statistical patterns in large text corpora rather than from a hand-written grammar.

3. Prerequisites

- Probability, conditional probability, logarithms, and cross-entropy
- Tokenization, embeddings, and vector similarity
- Neural networks, backpropagation, and gradient descent
- RNNs at a conceptual level and Transformers in detail
- PyTorch tensors, batching, masking, and GPU training

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Common interview angle
Tokenization	Converts text to token IDs	Defines sequence length and vocabulary	playing -> play, ing	Why subwords beat words
Autoregressive LM	Predicts the next token from a prefix	Natural fit for generation	`P(cat the)`	
Masked LM	Predicts deliberately hidden tokens	Learns bidirectional representations	Paris is the [MASK]	BERT vs GPT
Context window	Maximum tokens visible at once	Limits usable history and cost	8K tokens	Quadratic attention cost

Concept	Meaning	Why it matters	Simple example	Common interview angle
Pretraining	Self-supervised learning on large corpora	Produces general language capability	Next-token prediction	Why labels are unnecessary
Adaptation	Specializes a pretrained LM	Makes models useful for tasks	SFT, LoRA, prompting	Fine-tuning vs RAG
Perplexity	Exponentiated average NLL	Measures predictive uncertainty	Lower is better	Limits across tokenizers

5. Algorithm / Working Process

1. Normalize and tokenize text; prepend or append special tokens when required.
2. Map token IDs to embeddings and add position information.
3. Pass embeddings through Transformer blocks containing causal self-attention and feed-forward layers.
4. Project each hidden state to vocabulary logits.
5. During training, shift targets by one position and minimize next-token cross-entropy.
6. During inference, repeatedly sample or select one token, append it, and reuse the key-value cache.

Input: token IDs and an attention mask. **Output:** a probability distribution over the vocabulary at every position. **Training:** teacher forcing exposes the true previous tokens. **Inference:** the model consumes its own generated tokens, which can accumulate errors.

6. Mathematical Foundation

Autoregressive factorization turns a sequence probability into next-token probabilities:

$$P(x_{1:T}) = \prod_{t=1}^T P(x_t \mid x_{<t}).$$

Given logits z_t , softmax produces token probabilities:

$$p_{t,j} = \frac{e^{z_{t,j}}}{\sum_{k=1}^{|V|} e^{z_{t,k}}}.$$

The negative log-likelihood/cross-entropy loss is

$$\mathcal{L}_{LM} = -\frac{1}{T} \sum_{t=1}^T \log P_{\theta}(x_t \mid x_{<t}).$$

Perplexity is $\text{PPL} = \exp(\mathcal{L}_{LM})$. A perplexity of 20 loosely means the model is as uncertain as choosing among 20 equally likely tokens, but comparisons are valid only under comparable tokenization and data.

Self-attention is

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^{\top}}{\sqrt{d_k}} + M\right) V,$$

where causal mask M sets future positions to $-\infty$. AdamW commonly optimizes parameters using gradients of the LM loss, warmup, learning-rate decay, weight decay, and gradient clipping.

7. Practical Implementation

```
import torch
import torch.nn as nn

class TinyLanguageModel(nn.Module):
    def __init__(self, vocab_size: int, d_model: int = 64):
        super().__init__()
        self.embedding = nn.Embedding(vocab_size, d_model)
        self.rnn = nn.GRU(d_model, d_model, batch_first=True)
        self.head = nn.Linear(d_model, vocab_size)

    def forward(self, token_ids):
        hidden, _ = self.rnn(self.embedding(token_ids))
        return self.head(hidden)

vocab_size = 100
model = TinyLanguageModel(vocab_size)
optimizer = torch.optim.AdamW(model.parameters(), lr=3e-4)

# Four toy sequences; input predicts the token one step to its right.
batch = torch.randint(0, vocab_size, (4, 17))
inputs, targets = batch[:, :-1], batch[:, 1:]
logits = model(inputs)
loss = nn.functional.cross_entropy(
    logits.reshape(-1, vocab_size), targets.reshape(-1)
)
optimizer.zero_grad()
loss.backward()
optimizer.step()
print(f"loss={loss.item():.3f}, perplexity={loss.exp().item():.2f}")
```

8. Code Explanation

Embedding converts IDs to dense vectors. The GRU is used only to expose the LM training contract compactly; production LMs use causal Transformers. The input/target shift implements teacher forcing. Flattening logits and targets lets cross-entropy treat every non-padding position as a classification example. Real training must mask padding labels, usually by assigning them `ignore_index=-100`.

9. Training / Evaluation

Prepare deduplicated, licensed, quality-filtered text; split by document or source to prevent near-duplicate leakage. Track validation NLL/perplexity, downstream task accuracy, factuality, safety, calibration, latency, and human preference. Important hyperparameters include parameter count, context length, tokens per batch, learning rate, warmup, weight decay, and training-token budget. Reduce overfitting with more diverse data, regularization, early stopping for smaller models, and contamination audits.

10. Complexity and Cost

For vanilla attention, training compute per layer is roughly $O(T^2d + Td^2)$ and attention memory is $O(T^2)$. Autoregressive decoding without a cache repeatedly recomputes the prefix; a key-value cache reduces repeated compute but consumes approximately $O(LTd)$ memory across layers. Large-model training requires distributed GPUs, mixed precision, gradient accumulation/checkpointing, and careful communication. Quantization reduces inference memory and sometimes latency.

11. Common Use Cases

- Chatbots, copilots, and code completion
- Summarization, translation, rewriting, and classification
- Semantic representations and retrieval
- Structured extraction and synthetic data generation
- Tool-using agents and natural-language interfaces

12. Common Mistakes

- Comparing perplexity across different tokenizers
- Forgetting the causal or padding mask
- Splitting randomly at chunk level and leaking a document into validation
- Treating fluent output as factual knowledge
- Using next-token accuracy alone for an open-ended product
- Fine-tuning when prompting or retrieval would solve the requirement
- Evaluating on benchmarks present in pretraining data

13. Edge Cases / Limitations

LMs can hallucinate, reproduce bias or private memorized strings, fail outside their training distribution, and be sensitive to prompt wording. A fixed context window does not create persistent memory. Next-token likelihood does not guarantee truth, reasoning correctness, or alignment with user intent. Rare languages and specialized domains may be poorly represented.

14. Variations

Variation	What changes	When to use	Importance
Causal LM	Left-to-right mask	Free-form generation	Essential for placements
Masked LM	Random tokens hidden	Encoding/classification	Essential comparison
Encoder-decoder LM	Encoder reads input; decoder generates	Translation/summarization	Important
Prefix LM	Prefix is bidirectional; output causal	Conditioned generation	Useful research concept

Variation	What changes	When to use	Importance
Mixture of Experts	Activates a subset of FFN experts	Scale parameters efficiently	Advanced interviews
State-space LM	Replaces/reduces attention recurrence	Very long sequences	Research-relevant

15. Related Topics

- **RNN vs Transformer:** RNNs process sequentially; Transformers parallelize training and model long-range interactions directly.
- **Fine-tuning vs RAG:** fine-tuning changes behavior/weights; RAG injects current external knowledge at inference.
- **LoRA vs full fine-tuning:** LoRA trains small low-rank adapters with much lower memory.
- **Language modeling vs text generation:** the LM estimates probabilities; a decoding algorithm turns them into text.
- **Instruction tuning vs preference alignment:** SFT imitates demonstrations; DPO/RLHF optimizes preferences.

16. Interview Questions

1. **What does a language model learn?** A conditional distribution over tokens, usually $P(x_t \mid x_{<t})$, whose product defines sequence likelihood.
2. **Why is cross-entropy used?** It is the negative log-likelihood for categorical next-token targets and gives a differentiable maximum-likelihood objective.
3. **BERT vs GPT?** BERT uses bidirectional masked-token prediction for representations; GPT uses causal next-token prediction for generation.
4. **What is teacher forcing?** Training the model with ground-truth previous tokens instead of its sampled predictions.
5. **Why can exposure bias occur?** At inference the model conditions on its own possibly wrong outputs, a situation absent during teacher-forced training.
6. **What does perplexity miss?** Factuality, usefulness, safety, long-form coherence, and human preference; it also depends on tokenization.
7. **Why scale attention by $\sqrt{d_k}$?** It prevents dot products from growing with dimension and pushing softmax into saturated, low-gradient regions.
8. **What is a KV cache?** Stored attention keys and values for past tokens so decoding computes only the new position.
9. **Why does an LM hallucinate?** Its objective rewards probable continuations, not verified truth; uncertain prompts may still demand an answer.
10. **How would you evaluate a domain LM?** Use held-out domain perplexity plus task accuracy, factuality, safety, calibration, human evaluation, and leakage checks.
11. **Why are decoder-only models popular?** One simple scalable objective and architecture supports pretraining, in-context learning, and arbitrary continuation.
12. **What is data contamination?** Train data overlaps with evaluation examples, inflating benchmark results without genuine generalization.

17. Practice Tasks

- **Coding:** train a character-level LM and sample text at several temperatures.
- **Dataset project:** fine-tune a small causal LM on a licensed domain corpus.
- **Experiment:** plot validation perplexity against context length and model size.
- **Debugging:** find a target-shift or padding-mask bug in an LM training loop.
- **Extension:** add a KV cache and benchmark tokens per second.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Domain autocomplete	Completes technical notes	PyTorch, Transformers	Public domain manuals	Shows fine-tuning/evaluation
Tiny LM from scratch	Implements tokenizer and decoder	PyTorch	TinyStories	Demonstrates fundamentals
LM evaluation dashboard	Compares PPL, quality, latency	Python, Streamlit	Curated prompts	Shows production judgment

19. Quick Revision

- **Key idea:** model token-sequence probability through conditional prediction.
- **Main formula:** $P(x_{1:T}) = \prod_t P(x_t \mid x_{<t})$.
- **When to use:** any system that interprets or generates language.
- **Metrics:** NLL, perplexity, task metrics, factuality, safety, latency.
- **Common traps:** leakage, wrong masks, tokenizer mismatch, equating fluency with truth.
- **Interview one-liner:** "A causal LM is a maximum-likelihood next-token predictor whose learned distribution becomes a generator through decoding."

20. Final Cheat Sheet

Item	Summary
Definition	Probability model over token sequences
Input/output	Prefix token IDs → vocabulary logits
Main steps	Tokenize → embed → Transformer → logits → softmax
Key hyperparameters	Layers, width, heads, context, LR, batch tokens
Metrics	NLL/PPL plus task, human, safety, and latency metrics
Pros	General, scalable, transferable
Cons	Expensive, hallucination, bias, limited context
Best use cases	Language understanding, generation, and interfaces

Text Generation

1. Overview

Text generation converts a prompt or structured condition into a token sequence. A trained LM supplies next-token probabilities; a decoding strategy controls which tokens become the final answer. It is used in chat, code generation, stories, summarization, translation, data augmentation, report writing, and conversational search.

2. Intuition

Imagine walking through a tree where every branch is a possible next word. Greedy decoding always takes the widest branch, beam search keeps several paths, and sampling rolls a probability-weighted die. The underlying LM is unchanged; decoding changes diversity, repetition, determinism, and quality.

3. Prerequisites

- Autoregressive language models and tokenization
- Softmax, categorical sampling, entropy, and log probabilities
- Prompt formatting, BOS/EOS tokens, and context windows
- PyTorch or Hugging Face inference
- Basic evaluation and safety filtering

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Greedy decoding	Pick highest-probability token	Fast and deterministic	$\text{argmax}(p)$	Can be repetitive/myopic
Beam search	Keep top B partial sequences	Better sequence likelihood	Translation with beam 4	Length bias
Temperature	Rescales logits	Controls randomness	$T < 1$ sharper	Temperature is not top-p
Top-k	Sample only among k best tokens	Removes long tail	$k=50$	Fixed candidate count
Top-p	Smallest set with mass at least p	Adaptive candidate count	$p=0.9$	Often better for open text
Repetition control	Penalizes reused tokens/ngrams	Reduces loops	repetition penalty	Can damage factual terms
Stopping criteria	Ends generation safely	Controls latency/format	EOS or stop string	Token vs string boundary

5. Algorithm / Working Process

1. Format the prompt with the model's chat template and tokenize it.
2. Run a forward pass and extract logits at the last position.
3. Apply constraints, repetition penalties, temperature, and top-k/top-p filtering.
4. Select or sample the next token.

5. Append it, update the KV cache, and repeat until EOS, stop condition, or token budget.
6. Decode IDs, remove control tokens, validate structure, and apply product safety checks.

Training normally belongs to the underlying LM; generation is inference. For conditional tasks, supervised fine-tuning trains on (**instruction**, **response**) pairs with loss applied mainly to response tokens.

6. Mathematical Foundation

Temperature transforms logits before softmax:

$$P_T(x_i) = \frac{\exp(z_i/T)}{\sum_j \exp(z_j/T)}.$$

$T \rightarrow 0$ approaches argmax; larger T raises entropy. Top-k keeps indices in the k largest probabilities.

Nucleus sampling chooses the smallest sorted set S satisfying

$$\sum_{i \in S} P(x_i | x_{<t}) \geq p.$$

Beam search scores a sequence with summed log probabilities, often length-normalized:

$$s(y) = \frac{1}{|y|^\alpha} \sum_{t=1}^{|y|} \log P(y_t | y_{<t}, x).$$

Because log probabilities are non-positive, raw beam scores prefer short sequences; length normalization or a length penalty compensates.

7. Practical Implementation

```
from transformers import AutoModelForCausalLM, AutoTokenizer

name = "distilgpt2"
tokenizer = AutoTokenizer.from_pretrained(name)
model = AutoModelForCausalLM.from_pretrained(name)

prompt = "In a production text-generation service, monitoring should"
inputs = tokenizer(prompt, return_tensors="pt")
output = model.generate(
    **inputs,
    max_new_tokens=60,
    do_sample=True,
    temperature=0.8,
    top_p=0.92,
    repetition_penalty=1.1,
    pad_token_id=tokenizer.eos_token_id,
)
print(tokenizer.decode(output[0], skip_special_tokens=True))
```

8. Code Explanation

The tokenizer and model checkpoint must match. `max_new_tokens` bounds generated output independently of prompt length. `do_sample=True` enables stochastic decoding; otherwise temperature/top-p have no effect under greedy decoding. `top_p` truncates unlikely tail tokens and `repetition_penalty` discourages loops. Production code should batch requests, pass an attention mask, stream tokens, and record finish reasons.

9. Training / Evaluation

Evaluate according to the task: ROUGE for summarization, BLEU/COMET for translation, pass@k for code, exact match for constrained QA, and human preference/factuality for open generation. Also measure toxicity, schema validity, repetition, time to first token, inter-token latency, and cost. Use prompt-level or source-level splits and multiple random seeds because sampling is stochastic. Tune decoding on validation prompts, never on the test set.

10. Complexity and Cost

Generation is sequential: N new tokens require N decoding steps. With KV caching, per-step attention still grows with prefix length, and cache memory grows with batch size, layers, sequence length, heads, and head dimension. Beam width roughly multiplies decoding compute and cache memory. Batching raises throughput but may worsen individual latency; quantization reduces memory.

11. Common Use Cases

- Chat assistants and customer support drafts
- Code, SQL, test, and documentation generation
- Summarization and translation
- Personalized marketing or educational content
- Synthetic training examples and role-play simulation

12. Common Mistakes

- Setting both low temperature and restrictive top-p, destroying diversity
- Expecting temperature to matter when sampling is disabled
- Using beam search for creative dialogue and getting bland text
- Ignoring prompt tokens when enforcing the context limit
- Parsing unvalidated generated JSON directly
- Evaluating one random generation per prompt
- Applying stop strings without considering token boundaries

13. Edge Cases / Limitations

Generation may loop, stop prematurely, violate requested schemas, cite nonexistent sources, or follow malicious text inside retrieved content. Sampling makes outputs irreproducible unless seeds and hardware behavior are controlled. Long generation can drift from instructions. Hard lexical constraints may make all likely continuations invalid.

14. Variations

Variation	What changes	When to use	Importance
Greedy	One argmax path	Short factual completion	Basic
Beam search	Multiple high-score paths	Translation/transcription	Common interview topic
Contrastive search	Balances confidence and degeneration	Coherent long text	Advanced
Constrained decoding	Masks invalid tokens	JSON, grammar, SQL	Very useful in projects
Speculative decoding	Small draft model proposes tokens	Lower latency	Systems interviews
Retrieval-grounded generation	Adds retrieved evidence	Knowledge-intensive answers	Essential AI engineering

15. Related Topics

- **LM vs decoder:** the LM defines probabilities; the decoder makes a decision from them.
- **Top-k vs top-p:** top-k has fixed candidate count; top-p adapts to distribution sharpness.
- **Beam search vs sampling:** beam targets high likelihood; sampling targets representative diversity.
- **Prompting vs fine-tuning:** prompting supplies temporary instructions; fine-tuning changes model behavior.
- **Constrained decoding vs output repair:** constraints prevent invalid syntax; repair tries to fix it afterward.

16. Interview Questions

1. **Why is greedy decoding not globally optimal?** The locally best token can lead to a lower-probability complete sequence than another early choice.
2. **What does temperature do?** It divides logits before softmax, controlling entropy without changing their rank.
3. **Top-k vs top-p?** Top-k retains a fixed number of tokens; top-p retains a variable number whose cumulative mass reaches a threshold.
4. **Why can beam search prefer short outputs?** Every added token adds a non-positive log probability; length penalties correct this bias.
5. **Why is beam search uncommon for chat?** It favors high-likelihood, generic continuations and provides less natural diversity.
6. **What is repetition penalty?** A logit transformation that lowers probability for previously used tokens; it is a heuristic, not retraining.
7. **How do you produce reliable JSON?** Use grammar/schema-constrained decoding, low randomness, validation, and bounded retry or repair.
8. **What is speculative decoding?** A cheap model drafts several tokens and the target model verifies them while preserving the target distribution.
9. **How do you evaluate stochastic output?** Use multiple samples/seeds, distributional task metrics, human judgments, and safety checks.

10. **What causes high generation latency?** Sequential decoding, long prefixes, large KV caches, model size, memory bandwidth, and small inefficient batches.
11. **Does a lower temperature make facts correct?** No; it makes output more deterministic, not better grounded.
12. **How should a service stop output?** EOS, maximum new tokens, structured completion, explicit stop sequences, cancellation, and wall-clock limits.

17. Practice Tasks

- **Coding:** implement temperature, top-k, and top-p sampling from logits.
- **Dataset project:** build a headline generator from a public news summarization dataset.
- **Experiment:** compare diversity, repetition, and quality across decoding settings.
- **Debugging:** diagnose output that ignores top-p because `do_sample=False`.
- **Extension:** add JSON-schema-constrained output and validation.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Decoding playground	Visualizes token probabilities/settings	Transformers, Gradio	Curated prompts	Shows inference intuition
Grounded report writer	Generates answers with citations	RAG, FastAPI	Public documents	Production GenAI skills
Code-generation evaluator	Runs tests and reports pass@k	Python, sandboxing	HumanEval-style tasks	Evaluation and safety

19. Quick Revision

- **Key idea:** decoding turns LM probabilities into a sequence.
- **Main formula:** $P_T(i) = \text{softmax}(z_i/T)$.
- **When to use:** all autoregressive content creation.
- **Metrics:** task score, factuality, diversity, safety, latency, cost.
- **Common traps:** incompatible decoding flags, no length bound, unvalidated structure.
- **Interview one-liner:** "The model supplies logits; temperature and truncation shape the distribution, and the decoding policy selects the path."

20. Final Cheat Sheet

Item	Summary
Definition	Autoregressive construction of text from token probabilities
Input/output	Prompt tokens → generated tokens/text
Main steps	Forward → filter logits → choose token → append → stop
Key hyperparameters	max tokens, temperature, top-k, top-p, beams, penalties
Metrics	Quality, factuality, diversity, validity, latency

Item	Summary
Pros	Flexible, controllable, easy to prompt
Cons	Sequential cost, hallucination, decoding sensitivity
Best use cases	Chat, code, summarization, translation, drafting

Image Generation Basics

1. Overview

Image generation learns a distribution over images and samples new pixels or latent representations, optionally conditioned on labels, text, layouts, depth, or reference images. It supports creative tools, advertising, design ideation, synthetic training data, game assets, restoration, and scientific imaging. The major model families are autoregressive models, VAEs, GANs, diffusion models, and flow-based models.

2. Intuition

Suppose a model studies millions of dog images. It does not store one average dog; it learns which visual patterns tend to coexist—fur textures, eyes, poses, backgrounds—and can combine them into a new sample. Conditioning is like adding a brief: “a black dog running on snow.”

3. Prerequisites

- Images as tensors, RGB channels, normalization, and resizing
- CNNs, residual blocks, attention, and up/downsampling
- Probability distributions, likelihood, latent variables, and sampling
- Backpropagation, optimization, regularization, and EMA
- PyTorch, torchvision transforms, and GPU fundamentals

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Data distribution	Unknown distribution of real images	Generation approximates it	Face dataset	Modeling density vs samples
Latent code	Compact random/semantic representation	Controls variation	$z \sim N(0, I)$	Latent interpolation
Generator	Maps noise/condition to image	Creates samples	$G(z)$	Upsampling artifacts

Concept	Meaning	Why it matters	Simple example	Interview angle
Conditioning	Guides content	Makes output useful	class/text/depth	Injection mechanisms
Resolution hierarchy	Features at multiple scales	Global layout and fine texture differ	U-Net pyramid	Receptive field
Diversity vs fidelity	Coverage vs visual quality	A model can excel at only one	mode collapse	FID limitations
Normalization/range	Pixel scaling convention	Must match model/output	$[-1, 1]$	Tanh and denormalization

5. Algorithm / Working Process

1. Collect and filter images; create captions or labels if conditional generation is needed.
2. Decode, resize/crop, augment carefully, and normalize pixel tensors.
3. Sample randomness such as latent noise, diffusion noise, or an autoregressive token order.
4. Predict images directly, decode latents, or iteratively denoise depending on the family.
5. Compare predictions with training targets through reconstruction, adversarial, likelihood, or score-matching objectives.
6. At inference, sample a seed, apply the condition, run the generator/sampler, decode, and clamp pixels.

Input: random noise plus optional condition. **Processing:** learned synthesis in pixel or latent space. **Output:** an image tensor, usually converted to uint8 RGB.

6. Mathematical Foundation

A generator transforms a base random variable into an image:

$$z \sim p(z), \quad x = G_\theta(z, c),$$

where c is optional conditioning. Maximum-likelihood approaches minimize

$$\mathcal{L}_{NLL} = -\mathbb{E}_{x \sim p_{data}} \log p_\theta(x).$$

Pixel reconstruction often uses L_1 or L_2 :

$$\mathcal{L}_{rec} = \frac{1}{CHW} \|x - \hat{x}\|_1.$$

L_2 corresponds to a Gaussian observation model and often averages uncertain detail; perceptual loss compares pretrained feature maps:

$$\mathcal{L}_{perc} = \sum_l \|\phi_l(x) - \phi_l(\hat{x})\|_2^2.$$

FID compares Gaussian approximations to real and generated feature distributions:

$$FID = \|\mu_r - \mu_g\|_2^2 + \text{Tr}(\Sigma_r + \Sigma_g - 2(\Sigma_r \Sigma_g)^{1/2}).$$

Lower FID is better, but its estimate depends on sample count, feature extractor, preprocessing, and domain.

7. Practical Implementation

```
import torch
from torch import nn
from torchvision.utils import save_image

class SmallGenerator(nn.Module):
    def __init__(self, z_dim=64):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(z_dim, 128 * 7 * 7), nn.ReLU(),
            nn.Unflatten(1, (128, 7, 7)),
            nn.ConvTranspose2d(128, 64, 4, 2, 1), nn.BatchNorm2d(64), nn.ReLU(),
            nn.ConvTranspose2d(64, 1, 4, 2, 1), nn.Tanh(),
        )

    def forward(self, z):
        return self.net(z)

torch.manual_seed(7)
generator = SmallGenerator()
z = torch.randn(16, 64)
images = generator(z)  # (16, 1, 28, 28), values in [-1, 1]
save_image((images + 1) / 2, "generated_grid.png", nrow=4)
assert images.shape == (16, 1, 28, 28)
```

8. Code Explanation

The linear layer expands a latent vector into a small spatial feature map. Transposed convolutions double resolution from 7×7 to 14×14 and then 28×28 . `Tanh` matches training images normalized to `[-1, 1]`; the save step reverses that normalization. The example demonstrates synthesis only—training requires a family-specific objective such as GAN discrimination or reconstruction.

9. Training / Evaluation

Split by source, identity, scene, or video—not merely by randomly augmented images—to avoid leakage. Monitor FID/KID, precision and recall for fidelity/coverage, CLIP score for text alignment, LPIPS for perceptual diversity, and human preference. Check memorization with nearest-neighbor analysis. Hyperparameters include resolution, architecture width, batch size, learning rate, augmentation, noise schedule or latent size, EMA decay, and guidance strength.

10. Complexity and Cost

Cost grows quickly with pixel count: doubling both dimensions creates four times as many pixels. CNN activation memory is roughly proportional to batch $\times$ channels $\times$ height $\times$ width; global pixel attention is quadratic in the number of spatial tokens. GAN inference can require one forward pass, whereas diffusion uses many denoising evaluations. Latent-space generation substantially lowers spatial compute.

11. Common Use Cases

- Concept art, product mockups, and marketing assets
- Synthetic data for rare classes or privacy-aware prototyping
- Inpainting, outpainting, super-resolution, and restoration
- Avatars, game textures, and virtual try-on
- Scientific simulation, medical research, and anomaly modeling

12. Common Mistakes

- Mismatching input normalization and output activation
- Evaluating only attractive hand-picked samples
- Ignoring diversity or training-set memorization
- Randomly splitting near-duplicate frames across train and test
- Using FID computed with different preprocessing or sample counts
- Upsampling with settings that cause checkerboard artifacts
- Assuming more prompt guidance always improves quality

13. Edge Cases / Limitations

Models struggle with rare compositions, exact text, counting, hands, consistent geometry, very high resolution, and data-poor domains. Biased datasets produce biased outputs. Copyright, consent, provenance, and deepfake misuse matter independently of model quality. Synthetic data can reinforce model errors if used without real-data validation.

14. Variations

Variation	What changes	When to use	Importance
Autoregressive image model	Predicts pixel/image token sequence	Exact likelihood, discrete tokens	Research/advanced
VAE	Probabilistic encoder-decoder	Smooth latent representations	Foundational
GAN	Adversarial discriminator	Fast, sharp sampling	Essential interview topic
Diffusion	Iterative noise removal	High quality/diversity	Essential modern topic
Normalizing flow	Invertible transformations	Exact likelihood	Research
Latent diffusion	Denoises compressed latents	Efficient high resolution	Project-critical

15. Related Topics

- **Generation vs reconstruction:** generation samples unseen images; reconstruction tries to reproduce a given input.
- **GAN vs diffusion:** GANs sample quickly but can collapse; diffusion is slower but stable and diverse.

- **Pixel vs latent space:** pixels preserve all detail but cost more; latents compress redundant information.
- **Fidelity vs diversity:** sharp samples can still cover only a small part of the data distribution.
- **Conditional vs unconditional:** conditional models follow labels/text; unconditional models learn only the overall distribution.

16. Interview Questions

1. **What does image generation model?** A distribution $p(x)$ or conditional distribution $p(x | c)$ over image tensors.
2. **Why sample a latent vector?** It provides stochastic variation; the generator maps a simple base distribution into complex images.
3. **Why normalize images?** Stable optimization and consistency with the output activation and pretrained model convention.
4. **What is the fidelity-diversity trade-off?** A model may make very realistic samples while missing modes, or cover modes with lower individual quality.
5. **What does FID measure?** Distance between real and generated feature distributions approximated as Gaussians.
6. **Why is FID imperfect?** It depends on features/preprocessing/sample size, misses prompt alignment, and may not reflect human preference in specialized domains.
7. **What causes checkerboard artifacts?** Uneven overlap in transposed convolution; resize-convolution or careful kernel/stride choices help.
8. **Why generate in latent space?** Lower spatial dimensions reduce compute and memory while retaining perceptually important content.
9. **How do you test memorization?** Nearest-neighbor search against training data, duplicate detection, canary tests, and membership/privacy audits.
10. **Why can pixel MSE produce blur?** When several outputs are plausible, minimizing squared error predicts their conditional mean.
11. **How do you evaluate conditional generation?** Combine image quality/diversity with condition alignment and human task-specific judgment.
12. **Can synthetic data replace real test data?** No; evaluation must remain grounded in representative real-world data.

17. Practice Tasks

- **Coding:** build a decoder that maps noise to MNIST-sized images.
- **Dataset project:** train a class-conditional generator on Fashion-MNIST.
- **Experiment:** compare nearest-neighbor and transposed-convolution upsampling.
- **Debugging:** find a $[-1, 1]$ versus $[0, 1]$ mismatch causing washed-out samples.
- **Extension:** calculate FID/KID and perform a memorization audit.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Fashion asset generator	Produces category-controlled apparel	PyTorch, torchvision	Fashion-MNIST/DeepFashion subset	End-to-end generation

Project	What it does	Tech stack	Dataset	Resume value
Synthetic defect studio	Creates rare manufacturing defects	Diffusers, OpenCV	MVTec AD	Industrial CV relevance
Generator benchmark	Compares quality, diversity, speed	PyTorch, FID/KID	CIFAR-10	Strong evaluation story

19. Quick Revision

- **Key idea:** transform simple randomness, optionally plus a condition, into realistic images.
- **Main formula:** $x = G_{\theta}(z, c)$.
- **When to use:** synthesis, editing, augmentation, and restoration.
- **Metrics:** FID/KID, precision/recall, CLIP score, LPIPS, human preference.
- **Common traps:** normalization mismatch, cherry-picking, leakage, ignoring coverage.
- **Interview one-liner:** "Image generators approximate $p(x \mid c)$; architecture and objective decide the trade-off among likelihood, quality, diversity, and sampling speed."

20. Final Cheat Sheet

Item	Summary
Definition	Learned sampling of images from a data distribution
Input/output	Noise + optional condition → image tensor
Main steps	Prepare data → sample noise → synthesize → train objective → evaluate
Key hyperparameters	Resolution, latent size, LR, batch, guidance/sampling steps
Metrics	FID, KID, precision/recall, alignment, human preference
Pros	Creative synthesis, augmentation, controllable editing
Cons	Cost, bias, memorization, geometry/text errors
Best use cases	Creative tools, simulation, restoration, synthetic data

Autoencoders

1. Overview

An autoencoder (AE) is a neural network trained to reconstruct its input through a lower-dimensional or otherwise constrained representation. The encoder maps x to a latent code z ; the decoder maps z back to $\hat{x}$. Autoencoders are used for representation learning, compression, denoising, anomaly detection, dimensionality reduction, and as components of latent generative systems.

2. Intuition

Imagine explaining a photograph using only a short note and then asking another person to redraw it. The note must retain important structure and discard redundancy. The bottleneck latent vector is that note. Without a real bottleneck or regularization, the network may simply learn to copy.

3. Prerequisites

- Feed-forward networks, CNNs, activations, and backpropagation
- Linear algebra, dimensionality reduction, and PCA intuition
- MSE, binary cross-entropy, and perceptual similarity
- Image normalization and PyTorch data pipelines
- Train/validation splits and anomaly-detection metrics

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Encoder	$z = f_{\theta}(x)$	Extracts compressed features	image $\rightarrow$ 32 numbers	Deterministic mapping
Bottleneck	Capacity restriction	Prevents trivial copying	latent dimension 16	Undercomplete vs regularized
Decoder	$\hat{x} = g_{\phi}(z)$	Reconstructs input	32 numbers $\rightarrow$ image	Symmetric architecture not required
Reconstruction loss	Measures input-output mismatch	Main learning signal	pixel MSE	Why MSE blurs
Denoising AE	Reconstructs clean x from corrupted $\tilde{x}$	Learns robust structure	remove Gaussian noise	Target must remain clean
Sparse AE	Penalizes latent activation	Learns selective features	L1 penalty on z	Overcomplete bottleneck
Reconstruction anomaly score	High error indicates unusual sample	Enables one-class detection	defective part	Threshold calibration

5. Algorithm / Working Process

1. Normalize inputs and optionally corrupt them for denoising.
2. Encoder compresses each input into $z = f_{\theta}(x)$.
3. Decoder reconstructs $\hat{x} = g_{\phi}(z)$.
4. Compute reconstruction loss and optional regularization.
5. Backpropagate through both networks and update jointly.
6. At inference, use z as a feature, $\hat{x}$ as a restoration, or reconstruction error as an anomaly score.

Input: an image/vector and optionally a corrupted version. **Output:** reconstruction and latent code. Unlike a VAE, a standard AE does not learn a normalized probability distribution from which arbitrary latent samples are reliably decoded.

6. Mathematical Foundation

The basic model is

$$z = f_{\theta}(x), \quad \hat{x} = g_{\phi}(z).$$

For real-valued inputs, minimize

$$\mathcal{L}_{MSE} = \frac{1}{N} \sum_{i=1}^N \|x_i - g_{\phi}(f_{\theta}(x_i))\|_2^2.$$

For Bernoulli-scaled pixels, binary cross-entropy may be used:

$$\mathcal{L}_{BCE} = - \sum_j [x_j \log \hat{x}_j + (1 - x_j) \log(1 - \hat{x}_j)].$$

A sparse AE adds $\lambda \|z\|_1$. A denoising AE minimizes $\mathbb{E}_{\tilde{x} \sim q(\tilde{x}|x)} \ell(x, g(f(\tilde{x})))$. A linear undercomplete AE with MSE learns the same principal subspace as PCA, although its coordinates need not equal PCA coordinates.

7. Practical Implementation

```
import torch
from torch import nn

class Autoencoder(nn.Module):
    def __init__(self):
        super().__init__()
        self.encoder = nn.Sequential(nn.Flatten(), nn.Linear(28 * 28, 128),
nn.ReLU(), nn.Linear(128, 16))
        self.decoder = nn.Sequential(nn.Linear(16, 128), nn.ReLU(), nn.Linear(128,
28 * 28), nn.Sigmoid())

    def forward(self, x):
        z = self.encoder(x)
        return self.decoder(z).view(-1, 1, 28, 28), z

model = Autoencoder()
optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
clean = torch.rand(32, 1, 28, 28)
noisy = (clean + 0.2 * torch.randn_like(clean)).clamp(0, 1)

reconstruction, latent = model(noisy)
loss = nn.functional.mse_loss(reconstruction, clean)
optimizer.zero_grad(); loss.backward(); optimizer.step()
print(loss.item(), latent.shape)
```

8. Code Explanation

The encoder reduces 784 pixel values to 16 latent values. The decoder reverses the dimensional path and `Sigmoid` matches targets in `[0,1]`. Passing noisy input but computing loss against clean data makes this a denoising AE. In real image work, convolutional blocks retain spatial locality better than flattening.

9. Training / Evaluation

Fit preprocessing only on the training split. For reconstruction, report MSE/MAE, PSNR, SSIM, LPIPS, and visual comparisons. For anomaly detection, choose a threshold from validation normals and labeled anomalies if available; report AUROC and especially AUPRC when anomalies are rare. Track downstream probe quality of z . Tune latent dimension, corruption strength, regularization, depth, LR, and loss composition.

10. Complexity and Cost

Cost is one encoder plus one decoder pass during training; inference may use only the encoder. Dense layers cost $O(d_{in}d_{hidden})$; convolutional cost depends on kernel size and feature-map dimensions. A smaller bottleneck reduces storage but not necessarily encoder compute. AEs are much cheaper to sample/reconstruct than iterative diffusion models.

11. Common Use Cases

- Denoising images, signals, and tabular measurements
- Learning embeddings for clustering or visualization
- Compression and dimensionality reduction
- Industrial and medical anomaly detection
- Pretraining encoders and compressing images for latent diffusion

12. Common Mistakes

- Calling a deterministic AE a generative probabilistic model
- Using an oversized network with no bottleneck or regularizer
- Training and testing on duplicate images
- Using sigmoid output with `[-1,1]` targets
- Declaring every high reconstruction error anomalous without validation calibration
- Assuming anomalies must reconstruct badly; powerful AEs may reconstruct them well
- Reporting only pixel MSE despite perceptually poor output

13. Edge Cases / Limitations

An AE can learn identity mapping, create blurry reconstructions, encode nuisance details, and generalize well enough to reconstruct anomalies. Its latent space can be irregular and contain “holes,” so random z samples may decode to nonsense. Pixel loss is misaligned with human perception. Compression learned on one domain may fail after distribution shift.

14. Variations

Variation	What changes	When to use	Importance
Undercomplete AE	Latent smaller than input	Compression/features	Placement basic
Denoising AE	Corrupt input, clean target	Robust features/restoration	Important
Sparse AE	Penalize activations	Interpretable/selective codes	Interview useful
Contractive AE	Penalize encoder Jacobian	Local robustness	Advanced
Convolutional AE	Spatial convolutions	Images	Project standard
VQ-AE/VQ-VAE	Discrete codebook	Image/audio tokens	Modern generation

15. Related Topics

- **AE vs PCA:** linear undercomplete AE with MSE recovers a PCA subspace; nonlinear AEs learn curved manifolds.
- **AE vs VAE:** AE learns point codes; VAE learns posterior distributions regularized toward a prior.
- **AE vs U-Net:** both can encode/decode, but skip connections in a U-Net may bypass the bottleneck.
- **Reconstruction vs generation:** reconstruction conditions on an input; unconditional generation samples a learned distribution.
- **AE anomaly detection vs supervised classifier:** AE needs mostly normal data; a classifier needs representative anomaly labels.

16. Interview Questions

1. **What prevents an AE from copying input?** An undercomplete bottleneck, corruption, sparsity, contractive regularization, or architectural constraints.
2. **Why is a standard AE not a reliable generator?** Its latent codes have no enforced simple distribution, so random latent points may lie off the learned manifold.
3. **When does an AE resemble PCA?** A single linear undercomplete encoder-decoder trained with MSE spans the principal subspace.
4. **What is a denoising AE?** It receives corrupted input but reconstructs the original clean sample.
5. **Why can MSE make output blurry?** It favors the conditional mean when multiple sharp reconstructions are plausible.
6. **How is AE anomaly detection performed?** Train mainly on normal data, calculate reconstruction/latent score, and calibrate a threshold on validation data.
7. **Why may anomaly detection fail?** A high-capacity decoder may also reconstruct anomalies accurately.
8. **Undercomplete vs sparse AE?** Undercomplete restricts dimension; sparse AE can be overcomplete but restricts active units.
9. **Which output activation should be used?** Match data range and likelihood: sigmoid for $[0, 1]$, tanh for $[-1, 1]$, linear for unconstrained values.
10. **How do skip connections affect an AE?** They improve reconstruction but may let information bypass the latent bottleneck.
11. **How do you evaluate latent quality?** Linear probes, clustering, retrieval, interpolation, robustness, and downstream task metrics.
12. **What is the role of the decoder after deployment?** Keep it for reconstruction/generation; discard it if only embeddings are required.

17. Practice Tasks

- **Coding:** implement a convolutional AE for MNIST or CIFAR-10.
- **Dataset project:** detect manufacturing anomalies using reconstruction error.
- **Experiment:** sweep latent size and plot reconstruction vs probe accuracy.
- **Debugging:** diagnose outputs stuck near 0.5 because of normalization mismatch.
- **Extension:** combine pixel, SSIM, and perceptual losses.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Defect detector	Flags high reconstruction error	PyTorch, OpenCV	MVTec AD	Industrial anomaly pipeline
Document denoiser	Removes scan noise	Conv AE, FastAPI	Noisy Office/own pairs	Deployable CV system
Embedding explorer	Clusters learned representations	PyTorch, UMAP	Fashion-MNIST	Representation analysis

19. Quick Revision

- **Key idea:** encode to a constrained representation and reconstruct.
- **Main formula:** $\hat{x} = g_{\phi}(f_{\theta}(x))$ with reconstruction loss.
- **When to use:** compression, denoising, features, one-class anomaly detection.
- **Metrics:** MSE/MAE, PSNR/SSIM/LPIPS, AUROC/AUPRC.
- **Common traps:** identity mapping, output-range mismatch, assuming random latent sampling works.
- **Interview one-liner:** "An autoencoder learns useful structure only when its information path is constrained."

20. Final Cheat Sheet

Item	Summary
Definition	Encoder-decoder trained to reconstruct input
Input/output	Sample $x \rightarrow$ code z and reconstruction $\hat{x}$
Main steps	Encode $\rightarrow$ bottleneck $\rightarrow$ decode $\rightarrow$ reconstruction loss
Key hyperparameters	Latent size, depth, LR, corruption, regularization
Metrics	MSE, PSNR, SSIM, LPIPS; AUROC/AUPRC for anomalies
Pros	Simple, label-free, useful embeddings
Cons	Irregular latent space, blur, trivial copying
Best use cases	Denoising, compression, feature and anomaly learning

Variational Autoencoders

1. Overview

A Variational Autoencoder (VAE) is a latent-variable generative model. Its encoder approximates a distribution $q_\phi(z \mid x)$ rather than producing one point, and its decoder defines $p_\theta(x \mid z)$. Training maximizes an evidence lower bound (ELBO), balancing reconstruction with a KL penalty that organizes latent codes around a prior. VAEs support sampling, interpolation, representation learning, anomaly modeling, and latent compression.

2. Intuition

Instead of assigning every image a single address, a VAE assigns it a small neighborhood on a smooth map. Nearby neighborhoods represent similar images, while a rule encourages the entire map to resemble a standard Gaussian. You can then pick a coordinate from that Gaussian and decode a plausible new image.

3. Prerequisites

- Autoencoders and latent-variable models
- Gaussian distributions, expectation, variance, and KL divergence
- Conditional probability, Bayes' rule, and maximum likelihood
- Backpropagation and Monte Carlo estimation
- PyTorch tensor broadcasting and numerical stability

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Approximate posterior	$q_\phi(z \mid x)$	Replaces intractable true posterior	diagonal Gaussian	Amortized inference
Prior	Usually $p(z) = N(0, I)$	Gives a sampleable latent source	random normal	Prior mismatch
Reparameterization	$z = \mu + \sigma \odot \epsilon$	Makes sampling differentiable wrt parameters	$\epsilon \sim N(0, I)$	Core derivation
Reconstruction term	Expected log-likelihood	Preserves sample information	BCE/MSE	Likelihood choice
KL term	Aligns posterior and prior	Smooth, sampleable latent space	KL to unit Gaussian	Posterior collapse
ELBO	Tractable lower bound on log evidence	Training objective	reconstruction – KL	Why “lower bound”
Latent traversal	Vary one/few z coordinates	Inspects learned factors	pose changes	Disentanglement not guaranteed

5. Algorithm / Working Process

1. Encoder outputs $\mu_\phi(x)$ and $\log \sigma_\phi^2(x)$.
2. Sample $\epsilon \sim N(0, I)$ and compute $z = \mu + \sigma \odot \epsilon$.
3. Decoder predicts parameters of $p_\theta(x | z)$, such as Bernoulli probabilities or Gaussian means.
4. Compute reconstruction negative log-likelihood plus KL divergence to the prior.
5. Backpropagate through the reparameterized sample and update encoder/decoder.
6. At inference, reconstruct via encoded z , or generate by sampling $z \sim N(0, I)$ and decoding.

6. Mathematical Foundation

Marginal likelihood requires integrating latent variables:

$$p_\theta(x) = \int p_\theta(x | z)p(z) dz,$$

which is usually intractable. Variational inference introduces $q_\phi(z | x)$ and maximizes

$$\log p_\theta(x) \geq \mathcal{L}_{ELBO} = \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x | z)] - D_{KL}(q_\phi(z | x) || p(z)).$$

For diagonal $q = N(\mu, \text{diag}(\sigma^2))$ and $p = N(0, I)$,

$$D_{KL} = \frac{1}{2} \sum_j (\mu_j^2 + \sigma_j^2 - 1 - \log \sigma_j^2).$$

Reparameterization $z = \mu + \exp(0.5 \log \sigma^2) \odot \epsilon$ moves randomness into ϵ , allowing pathwise gradients. In minimization form: $\mathcal{L} = \mathcal{L}_{recon} + \beta D_{KL}$.

7. Practical Implementation

```
import torch
from torch import nn

class VAE(nn.Module):
    def __init__(self, latent_dim=16):
        super().__init__()
        self.enc = nn.Sequential(nn.Flatten(), nn.Linear(784, 256), nn.ReLU())
        self.mu, self.logvar = nn.Linear(256, latent_dim), nn.Linear(256,
latent_dim)
        self.dec = nn.Sequential(nn.Linear(latent_dim, 256), nn.ReLU(),
nn.Linear(256, 784))

    def forward(self, x):
        h = self.enc(x)
        mu, logvar = self.mu(h), self.logvar(h)
        z = mu + torch.exp(0.5 * logvar) * torch.randn_like(mu)
        logits = self.dec(z).view_as(x)
        return logits, mu, logvar

model = VAE()
x = torch.rand(32, 1, 28, 28)
logits, mu, logvar = model(x)
```

```
recon = nn.functional.binary_cross_entropy_with_logits(logits, x, reduction="sum")
/ len(x)
kl = -0.5 * (1 + logvar - mu.square() - logvar.exp()).sum() / len(x)
loss = recon + kl
loss.backward()
print({"loss": loss.item(), "reconstruction": recon.item(), "kl": kl.item()})
```

8. Code Explanation

The encoder shares a hidden representation but has separate heads for mean and log variance. Predicting `logvar` avoids constraining variance directly and improves numerical behavior. `randn_like` implements reparameterization. `binary_cross_entropy_with_logits` is more stable than applying sigmoid then BCE. Both loss terms use the same per-batch scaling, which is essential when interpreting β .

9. Training / Evaluation

Use representative, deduplicated splits. Track total ELBO, reconstruction term, KL per latent dimension, active units, FID/KID, sample diversity, interpolation smoothness, and downstream probes. A near-zero KL with a strong decoder signals posterior collapse. Mitigations include KL warmup/annealing, free bits, weaker decoder, richer posterior, or altered objective. Tune latent dimension, β , likelihood variance, architecture, and annealing schedule.

10. Complexity and Cost

Training costs one encoder and decoder pass with one or a few Monte Carlo samples; inference generation uses only the decoder and is fast. Memory resembles an AE with two encoder output heads. High-resolution pixel VAEs are expensive, so modern systems use convolutional encoders and perceptual/adversarial reconstruction objectives to learn smaller spatial latents.

11. Common Use Cases

- Smooth latent interpolation and controllable representation learning
- Generative sampling and data augmentation
- Anomaly detection via ELBO/reconstruction scores
- Molecule, speech, and recommendation latent-variable modeling
- Learned compression for latent diffusion models

12. Common Mistakes

- Getting the KL sign wrong
- Treating log variance as standard deviation
- Sampling directly from `logvar`
- Summing reconstruction but averaging KL inconsistently
- Using BCE for data outside `[0,1]` without a suitable likelihood
- Evaluating only reconstruction and not prior samples
- Calling latent dimensions disentangled without evidence

13. Edge Cases / Limitations

VAEs often generate smoother or blurrier images than adversarial/diffusion models because simple likelihoods average uncertainty. A flexible decoder may ignore z (posterior collapse). A unit Gaussian prior may be too simple; the aggregate posterior may not match it well. ELBO is a lower bound, and a better bound does not always mean better perceptual samples.

14. Variations

Variation	What changes	When to use	Importance
β -VAE	Weights KL by β	Representation/disentanglement trade-off	Common interview
Conditional VAE	Conditions encoder/decoder on c	Label-guided generation	Project useful
VQ-VAE	Discrete codebook, commitment loss	Image/audio tokenization	Modern essential
Hierarchical VAE	Multiple latent levels	Complex multi-scale data	Research
IWAE	Tighter multi-sample likelihood bound	Better density estimation	Advanced
Flow posterior	Flexible transformed posterior	Reduce variational gap	Research

15. Related Topics

- **VAE vs AE:** probabilistic posterior and KL regularization make VAE latents sampleable.
- **VAE vs GAN:** VAE has explicit likelihood bound and stable training; GAN often produces sharper samples.
- **VAE vs diffusion:** VAE usually generates in one decoder pass; diffusion iteratively denoises and often has higher fidelity.
- **KL vs reconstruction:** KL increases organization; reconstruction preserves instance detail.
- **VQ-VAE vs continuous VAE:** VQ-VAE uses discrete codes suitable for autoregressive modeling.

16. Interview Questions

1. **Why is the VAE “variational”?** It approximates an intractable posterior with a learned variational distribution optimized through a lower bound.
2. **What is the ELBO?** Expected reconstruction log-likelihood minus posterior-to-prior KL; it lower-bounds $\log p(x)$.
3. **Why use reparameterization?** It enables low-variance gradients through stochastic latent sampling.
4. **Why predict log variance?** It is unconstrained, numerically convenient, and converted to positive standard deviation by exponentiation.
5. **What does the KL term do?** It keeps each approximate posterior near the prior, creating a smoother, sampleable latent space.
6. **What is posterior collapse?** The decoder ignores z , causing $q(z \mid x)$ to match the prior and KL to approach zero.

7. **How can collapse be reduced?** KL warmup, free bits, decoder weakening, skip design changes, or more informative/hierarchical latents.
8. **Why are VAE outputs sometimes blurry?** Factorized Gaussian/Bernoulli reconstruction likelihoods and pixel losses average multiple plausible details.
9. **What does $\beta > 1$ do?** Stronger prior regularization can encourage separated factors but usually sacrifices reconstruction information.
10. **Can ELBO compare any two VAEs fairly?** Only when data preprocessing, likelihood definition, reduction, and evaluation protocol match.
11. **How do you generate after training?** Sample z from the prior and pass it through the decoder distribution.
12. **What is amortized inference?** One encoder learns to infer variational parameters for every input instead of optimizing them separately.

17. Practice Tasks

- **Coding:** implement a convolutional VAE and latent sampling grid.
- **Dataset project:** condition a VAE on Fashion-MNIST class labels.
- **Experiment:** sweep β and measure reconstruction, KL, and probe accuracy.
- **Debugging:** detect KL collapse from training curves and latent usage.
- **Extension:** replace the Gaussian latent with a VQ codebook.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Latent digit studio	Samples/interpolates controlled digits	PyTorch	MNIST	Shows ELBO and visualization
Molecular VAE	Generates candidate molecular strings/graphs	PyTorch, RDKit	QM9/ZINC subset	Research-oriented generation
VAE anomaly monitor	Scores unusual sensor windows	PyTorch, FastAPI	NASA turbofan/sensor data	Probabilistic deployment

19. Quick Revision

- **Key idea:** learn a sampleable latent distribution while reconstructing data.
- **Main formula:** ELBO = expected log-likelihood – KL.
- **When to use:** structured latent representations and fast probabilistic generation.
- **Metrics:** ELBO/NLL estimate, reconstruction, KL, FID, latent probes.
- **Common traps:** KL sign/scaling, collapse, mismatched observation likelihood.
- **Interview one-liner:** “A VAE makes an autoencoder generative by regularizing an approximate posterior toward a prior and training through reparameterization.”

20. Final Cheat Sheet

Item	Summary
Definition	Probabilistic encoder-decoder optimized via ELBO
Input/output	$x \rightarrow$ posterior parameters; $z \rightarrow$ generated/reconstructed x
Main steps	Encode $\mu, \log \sigma^2 \rightarrow$ sample $\rightarrow$ decode $\rightarrow$ recon + KL
Key hyperparameters	Latent size, β , likelihood, KL schedule, LR
Metrics	ELBO, reconstruction, KL, FID/KID, downstream quality
Pros	Stable, fast sampling, smooth latent space
Cons	Blur, posterior collapse, prior mismatch
Best use cases	Representation, interpolation, compression, anomaly modeling

GANs

1. Overview

A Generative Adversarial Network (GAN) trains a generator G to produce samples and a discriminator/critic D to distinguish real from generated data. Their adversarial game can yield sharp, fast samples without an explicit likelihood. GANs remain important for image synthesis, super-resolution, translation, face editing, and interview understanding of distribution matching.

2. Intuition

A counterfeiter improves fake notes while an inspector improves fake detection. Feedback from the inspector teaches the counterfeiter what realism requires. At equilibrium, generated and real distributions match and the inspector cannot do better than chance.

3. Prerequisites

- Neural networks, CNNs, gradient descent, and alternating optimization
- Binary classification and cross-entropy
- Probability distributions, divergences, and expectations
- Image normalization and stable training practices
- PyTorch autograd, `.detach()`, and optimizer state

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Generator	$G(z)$ maps noise to fake data	Learns implicit distribution	vector $\rightarrow$ face	No tractable density
Discriminator	Estimates real vs fake	Supplies learned training signal	real score	Optimal discriminator

Concept	Meaning	Why it matters	Simple example	Interview angle
Minimax game	Opposing objectives	Drives distribution matching	$\min_G \max_D$	Nash equilibrium
Non-saturating loss	Maximizes $\log D(G(z))$	Stronger early gradients	generator BCE to real	Standard practical loss
Mode collapse	Many z map to few outputs	Destroys diversity	same face repeatedly	Diagnosis/mitigation
Critic	Real-valued score in WGAN	Estimates Wasserstein objective	no sigmoid	Lipschitz constraint
Conditional GAN	Feeds class/text condition	Controllable output	generate digit 7	Projection discriminator

5. Algorithm / Working Process

1. Sample a real minibatch x and latent noise z .
2. Generate fake samples $\tilde{x} = G(z)$.
3. Update D to score real high and detached fake low.
4. Resample or reuse noise and update G so $D(G(z))$ scores as real.
5. Repeat alternating steps; optionally maintain an EMA copy of G for inference.
6. At inference, discard D , sample z , and run one generator pass.

Care is required to freeze or detach the correct graph during each update. Discriminator and generator learning rates, update ratio, regularization, and data augmentation strongly affect stability.

6. Mathematical Foundation

Original minimax GAN:

$$\min_G \max_D \mathbb{E}_{x \sim p_{data}} [\log D(x)] + \mathbb{E}_{z \sim p(z)} [\log(1 - D(G(z)))].$$

For fixed G , $D^*(x) = \frac{p_{data}(x)}{p_{data}(x) + p_g(x)}$. Substitution shows the generator minimizes a Jensen–Shannon-divergence-related objective. In practice, the non-saturating generator minimizes

$$\mathcal{L}_G = -\mathbb{E}_z \log D(G(z)),$$

which gives stronger gradients. WGAN uses

$$\min_G \max_{\|D\|_L \leq 1} \mathbb{E}_{x \sim p_{data}} D(x) - \mathbb{E}_z D(G(z)),$$

with a 1-Lipschitz critic, often encouraged by gradient penalty $\lambda(\|\nabla_{\hat{x}} D(\hat{x})\|_2 - 1)^2$.

7. Practical Implementation

```

import torch
from torch import nn

G = nn.Sequential(nn.Linear(32, 128), nn.ReLU(), nn.Linear(128, 784), nn.Tanh())
D = nn.Sequential(nn.Linear(784, 128), nn.LeakyReLU(0.2), nn.Linear(128, 1))
g_opt = torch.optim.Adam(G.parameters(), 2e-4, betas=(0.5, 0.999))
d_opt = torch.optim.Adam(D.parameters(), 2e-4, betas=(0.5, 0.999))

real = torch.rand(64, 784) * 2 - 1
z = torch.randn(64, 32)

# Discriminator update
fake = G(z)
d_loss = nn.functional.softplus(-D(real)).mean() +
nn.functional.softplus(D(fake.detach())).mean()
d_opt.zero_grad(); d_loss.backward(); d_opt.step()

# Generator update (non-saturating logistic loss)
g_loss = nn.functional.softplus(-D(G(torch.randn(64, 32))))).mean()
g_opt.zero_grad(); g_loss.backward(); g_opt.step()
print({"d_loss": d_loss.item(), "g_loss": g_loss.item()})

```

8. Code Explanation

Images are scaled to $[-1, 1]$ to match the generator's `Tanh`. `softplus(-D(real))` and `softplus(D(fake))` are numerically stable logistic losses on raw discriminator logits. `fake.detach()` prevents the discriminator update from changing generator parameters. The generator then receives gradients through a fresh fake batch while trying to raise discriminator scores.

9. Training / Evaluation

Use diverse, deduplicated data and identity/source-aware splits. Monitor sample grids from fixed seeds, FID/KID, precision/recall, discriminator behavior, gradient norms, and latent interpolation. Loss values alone do not correlate reliably with image quality. Improve training with spectral normalization, gradient penalties, balanced update rates, DiffAugment/ADA for limited data, EMA, and established architectures such as StyleGAN.

10. Complexity and Cost

Training requires both networks and alternating backward passes, but generation requires one G forward pass and is usually far faster than diffusion. Memory is dominated by feature maps and optimizer states. High resolution is expensive and sensitive to batch size. There is no sequential token/denoising loop unless the generator architecture itself introduces one.

11. Common Use Cases

- Photorealistic face and avatar synthesis
- Super-resolution and image restoration
- Paired/unpaired image-to-image translation

- Style transfer, attribute editing, and domain adaptation
- Synthetic data and privacy research

12. Common Mistakes

- Forgetting `detach()` during the discriminator update
- Applying sigmoid twice or mixing logits with probability losses
- Judging training from adversarial losses alone
- Letting the discriminator overpower the generator
- Ignoring diversity and reporting only best samples
- Using batch normalization carelessly with tiny batches
- Assuming mode collapse is fixed by simply adding more epochs

13. Edge Cases / Limitations

GAN training can oscillate, diverge, suffer vanishing gradients, or collapse modes. It offers no straightforward normalized likelihood. Performance drops with small or imbalanced data unless regularized carefully.

Conditional GANs can ignore conditions. Generated samples may memorize identities, and training stability is more sensitive than for many reconstruction or diffusion objectives.

14. Variations

Variation	What changes	When to use	Importance
DCGAN	Convolutional architectural rules	Educational image generation	Placement basic
cGAN	Adds labels/conditions	Controlled synthesis	Essential
WGAN-GP	Wasserstein critic + gradient penalty	More meaningful gradients	Essential interview
Pix2Pix	Paired conditional GAN + L1	Paired translation	Project important
CycleGAN	Cycle consistency, unpaired domains	Unpaired translation	Common interview
StyleGAN	Style-modulated synthesis	High-quality faces/assets	Advanced/project

15. Related Topics

- **GAN vs VAE:** GANs favor sharp implicit samples; VAEs optimize an explicit variational likelihood bound.
- **GAN vs diffusion:** GANs have fast one-pass inference; diffusion usually offers easier training and better coverage.
- **Discriminator vs classifier:** a discriminator's negative class evolves as the generator changes.
- **JS vs Wasserstein:** Wasserstein distance can provide useful gradients when supports do not overlap.
- **Pix2Pix vs CycleGAN:** Pix2Pix needs aligned pairs; CycleGAN uses unpaired sets and cycle consistency.

16. Interview Questions

1. **What is the GAN objective?** A minimax game in which D separates real/fake and G makes generated samples indistinguishable.
2. **Why use non-saturating generator loss?** The original minimax generator can have vanishing gradients when D confidently rejects early fakes.

3. **What is mode collapse?** Distinct latent inputs map to too few output modes, producing low diversity.
4. **How do you detect collapse?** Repeated samples, poor recall, low latent sensitivity, class-coverage tests, and nearest-neighbor analysis.
5. **Why does WGAN help?** Wasserstein distance changes smoothly as distributions move and can supply informative critic gradients.
6. **Why enforce Lipschitz continuity?** Kantorovich–Rubinstein duality requires a 1-Lipschitz critic for the Wasserstein objective.
7. **What does `detach()` do?** Stops autograd from propagating the discriminator loss into G during the D update.
8. **Why can't loss alone select the best GAN?** Adversarial losses depend on both changing players and do not directly measure perceptual quality or coverage.
9. **What is feature matching?** Train G to match intermediate discriminator feature statistics, which can stabilize and improve coverage.
10. **How is a conditional GAN built?** Inject condition into G and D , through concatenation, conditional normalization, or projection.
11. **What happens at ideal equilibrium?** $p_g = p_{data}$ and the optimal discriminator returns $1/2$ everywhere.
12. **Why are GANs still useful?** Their one-pass generators offer excellent latency and mature specialized image translation/super-resolution models.

17. Practice Tasks

- **Coding:** train a DCGAN on Fashion-MNIST with fixed-seed sample grids.
- **Dataset project:** build a conditional GAN for balanced class augmentation.
- **Experiment:** compare BCE GAN and WGAN-GP stability and coverage.
- **Debugging:** find graph leakage caused by missing `detach()`.
- **Extension:** add spectral normalization and EMA; quantify the change in FID.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Product recoloring	Translates product styles	Pix2Pix, PyTorch	Facades/custom paired data	Conditional translation
Rare-class augments	Synthesizes minority examples	cGAN, scikit-learn	Imbalanced image set	Links generation to outcomes
GAN stability lab	Compares objectives/regularizers	PyTorch, W&B/MLflow	CIFAR-10	Research experimentation

19. Quick Revision

- **Key idea:** generator learns through an adversarially trained realism signal.
- **Main formula:** $\min_G \max_D E \log D(x) + E \log(1 - D(G(z)))$.
- **When to use:** sharp images and very fast generation/translation.
- **Metrics:** FID/KID, precision/recall, diversity, task/human scores.
- **Common traps:** collapse, unstable balance, detach/logit errors, cherry-picking.

- **Interview one-liner:** "A GAN matches distributions through a two-player game rather than an explicit likelihood."

20. Final Cheat Sheet

Item	Summary
Definition	Adversarial generator-discriminator framework
Input/output	Noise/condition → generated sample
Main steps	Generate fake → train D → train G → alternate
Key hyperparameters	LR, betas, update ratio, latent size, regularization
Metrics	FID/KID, precision/recall, diversity, latency
Pros	Sharp samples, one-pass fast inference
Cons	Unstable, mode collapse, no tractable likelihood
Best use cases	Image synthesis, translation, super-resolution

Diffusion Models

1. Overview

Diffusion models generate data by learning to reverse a gradual noising process. During training, clean data is corrupted at random noise levels and a neural network learns to predict the added noise, clean sample, velocity, or score. During inference, generation starts from Gaussian noise and iteratively denoises. Diffusion models dominate high-quality image generation and extend to audio, video, 3D, molecules, and control problems.

2. Intuition

Imagine repeatedly adding ink to a photograph until it becomes static. If a model learns what a slightly less corrupted photograph should look like at every stage, it can begin with static and reverse the process into a coherent new image. Training can jump directly to a random corruption level; inference must follow a numerical path back.

3. Prerequisites

- Gaussian distributions, conditional probability, and KL divergence
- Markov chains and basic stochastic processes
- CNN/U-Net, residual blocks, normalization, and attention
- Score functions $\nabla_x \log p(x)$ and denoising intuition
- PyTorch, random timesteps, EMA, and mixed precision

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Forward process	Fixed gradual Gaussian corruption	Creates supervised noisy-clean pairs	$x_0 \rightarrow x_T$	Closed-form sampling
Reverse process	Learned denoising transitions	Generates data	$x_T \rightarrow x_0$	Gaussian parameterization
Noise schedule	Sequence β_t or SNR	Controls corruption difficulty	linear/cosine	Schedule effects
Timestep embedding	Represents current noise level	One network handles all levels	sinusoidal embedding	Injected into U-Net blocks
Prediction target	ϵ , x_0 , or v	Affects weighting/stability	epsilon prediction	Convert among targets
Sampler	Numerical reverse trajectory	Controls speed/quality	DDPM, DDIM, DPM-Solver	Training vs sampler
Guidance	Steers conditional probability	Improves alignment	classifier-free guidance	Quality-diversity trade-off

5. Algorithm / Working Process

Training:

1. Sample clean x_0 , timestep t , and Gaussian noise ϵ .
2. Construct $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$ in one step.
3. Pass x_t , t , and optional condition to a U-Net/Transformer.
4. Compare prediction with the chosen target, commonly ϵ .
5. Update parameters; maintain EMA weights for evaluation.

Inference: sample $x_T \sim N(0, I)$, predict at decreasing timesteps, use scheduler equations to obtain x_{t-1} , and finish at x_0 . The output is stochastic unless using a deterministic sampler and fixed seed.

6. Mathematical Foundation

Define $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$. The forward transition is

$$q(x_t | x_{t-1}) = N(\sqrt{\alpha_t}x_{t-1}, \beta_t I),$$

and its closed form is

$$q(x_t | x_0) = N(\sqrt{\bar{\alpha}_t}x_0, (1 - \bar{\alpha}_t)I).$$

Thus $x_t = \sqrt{\bar{\alpha}_t}x_0 + \sqrt{1 - \bar{\alpha}_t}\epsilon$. A common simplified objective is

$$\mathcal{L}_{simple} = \mathbb{E}_{x_0, t, \epsilon} [\|\epsilon - \epsilon_\theta(x_t, t, c)\|_2^2].$$

The reverse distribution is modeled as $p_\theta(x_{t-1} | x_t) = N(\mu_\theta(x_t, t), \Sigma_t)$. From predicted noise,

$$\hat{x}_0 = \frac{x_t - \sqrt{1 - \bar{\alpha}_t} \epsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}.$$

Classifier-free guidance combines conditional and unconditional predictions:

$$\hat{e} = \epsilon_\theta(x_t, t, \emptyset) + w[\epsilon_\theta(x_t, t, c) - \epsilon_\theta(x_t, t, \emptyset)].$$

7. Practical Implementation

```
import torch
from torch import nn

class NoisePredictor(nn.Module):
    def __init__(self, steps=1000, hidden=256):
        super().__init__()
        self.time = nn.Embedding(steps, hidden)
        self.net = nn.Sequential(nn.Linear(784 + hidden, 512), nn.SiLU(),
nn.Linear(512, 784))

    def forward(self, x_t, t):
        flat = x_t.flatten(1)
        return self.net(torch.cat([flat, self.time(t)], dim=1)).view_as(x_t)

steps = 1000
beta = torch.linspace(1e-4, 0.02, steps)
alpha_bar = torch.cumprod(1 - beta, dim=0)
model = NoisePredictor(steps)

x0 = torch.rand(32, 1, 28, 28) * 2 - 1
t = torch.randint(0, steps, (len(x0),))
noise = torch.randn_like(x0)
a = alpha_bar[t].view(-1, 1, 1, 1)
x_t = a.sqrt() * x0 + (1 - a).sqrt() * noise
predicted_noise = model(x_t, t)
loss = nn.functional.mse_loss(predicted_noise, noise)
loss.backward()
print(loss.item())
```

8. Code Explanation

`alpha_bar` stores cumulative signal retention. Indexing it by a different random timestep per example creates all noise levels without simulating earlier steps. The learned timestep embedding tells the network how corrupted the input is. This compact MLP illustrates the correct objective; image systems use U-Nets or diffusion Transformers and a scheduler for inference.

9. Training / Evaluation

Use diverse, caption-cleaned, deduplicated data and source-aware held-out sets. Track denoising loss by timestep/SNR, FID/KID, precision/recall, CLIP alignment for text conditions, human preference, memorization, and safety. Tune noise/SNR schedule, prediction target, loss weighting, resolution, LR, batch size, EMA decay,

condition dropout, sampler, steps, and guidance. Validation should use fixed seeds/prompts plus sufficiently large unbiased samples.

10. Complexity and Cost

Training samples one timestep and usually requires one network evaluation per item, but large U-Nets/Transformers and high-resolution activations are expensive. Inference requires tens to hundreds of network evaluations, so it is slower than GAN/VAE decoding. Pixel attention can be costly; latent diffusion lowers spatial size. Distillation, consistency models, solver improvements, quantization, and caching reduce latency.

11. Common Use Cases

- Text-to-image and controlled image editing
- Inpainting, outpainting, restoration, and super-resolution
- Video, speech, music, and 3D asset generation
- Molecule/protein design and scientific inverse problems
- Synthetic data and conditional simulation

12. Common Mistakes

- Confusing α_t with cumulative $\bar{\alpha}_t$
- Broadcasting timestep coefficients incorrectly
- Mixing scheduler conventions or timestep order
- Training on one prediction type while sampling as another
- Omitting condition dropout but expecting classifier-free guidance
- Comparing FID from too few samples
- Increasing guidance until diversity and saturation collapse

13. Edge Cases / Limitations

Sampling is iterative and costly. Models can struggle with exact layout, counting, text, rare concepts, and global consistency. Strong guidance can oversaturate images and reduce diversity. Dataset bias, memorization, and unsafe synthesis remain concerns. A lower denoising MSE does not always translate monotonically to human-perceived quality.

14. Variations

Variation	What changes	When to use	Importance
DDPM	Stochastic ancestral sampling	Foundational understanding	Essential
DDIM	Non-Markovian, optionally deterministic path	Faster sampling/inversion	Essential
Score SDE	Continuous-time stochastic formulation	Theory/research	Advanced
Latent diffusion	Denoises autoencoder latents	Efficient high resolution	Essential

Variation	What changes	When to use	Importance
Diffusion Transformer	Transformer denoiser over patches/latents	Scalable modern systems	Research/project
Consistency/distilled model	Learns few-step mapping	Low-latency generation	Systems/research

15. Related Topics

- **Diffusion vs score matching:** the noise predictor is related to the score of noisy data distributions.
- **DDPM vs DDIM:** DDPM is stochastic ancestral sampling; DDIM can follow a deterministic trajectory with fewer steps.
- **Diffusion vs flow matching:** diffusion learns stochastic/noise reversal; flow matching learns a continuous velocity field.
- **Pixel vs latent diffusion:** latent models trade fine information for major compute savings.
- **CFG vs classifier guidance:** CFG trains a single conditional/unconditional model; classifier guidance needs a noise-aware classifier.

16. Interview Questions

1. **Why add noise during training?** It creates known corruption levels and teaches a model the local direction back toward the data distribution.
2. **Why can x_t be sampled directly?** Gaussian transitions compose into a Gaussian with cumulative coefficient $\bar{\alpha}_t$.
3. **What does the network predict?** Common targets are added noise ϵ , clean sample x_0 , velocity v , or score.
4. **Why is timestep conditioning required?** The correct denoising operation depends on current signal-to-noise ratio.
5. **Training vs sampling cost?** Training usually uses one random timestep per example; inference integrates many reverse steps.
6. **What is classifier-free guidance?** Interpolate/extrapolate between unconditional and conditional predictions from one condition-dropout-trained model.
7. **What happens when guidance is too high?** Better prompt adherence may turn into oversaturation, artifacts, and reduced diversity.
8. **Why use EMA weights?** A smoothed parameter trajectory often yields more stable, higher-quality samples.
9. **What is DDIM inversion?** Map an image approximately into a deterministic diffusion latent trajectory for reconstruction/editing.
10. **Why does latent diffusion help?** It performs repeated denoising on a much smaller spatial representation.
11. **What is SNR weighting?** Reweight losses across noise levels so easy or high-SNR steps do not dominate learning.
12. **Can diffusion compute exact likelihood?** Variational bounds or probability-flow ODEs can estimate likelihood, but image systems are usually optimized/evaluated for sample quality.

17. Practice Tasks

- **Coding:** train a tiny DDPM on MNIST and implement ancestral sampling.
- **Dataset project:** build class-conditional CIFAR-10 diffusion.
- **Experiment:** compare linear/cosine schedules and epsilon/v prediction.
- **Debugging:** identify an incorrect `alpha_bar` broadcast or reversed timestep loop.
- **Extension:** implement classifier-free guidance and plot fidelity-diversity changes.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
DDPM from scratch	Trains/samples handwritten digits	PyTorch	MNIST	Strong mathematical foundation
Restoration diffusion	Removes structured corruption	PyTorch, OpenCV	CelebA/CIFAR corruptions	Conditional inverse problems
Sampler benchmark	Compares quality vs NFE/latency	Diffusers, MLflow	Fixed prompt suite	Production evaluation

19. Quick Revision

- **Key idea:** learn to reverse a known noising process.
- **Main formula:** $x_t = \sqrt{\alpha_t}x_0 + \sqrt{1 - \alpha_t}\epsilon$.
- **When to use:** high-quality diverse synthesis and conditional editing.
- **Metrics:** FID/KID, precision/recall, alignment, NFE, latency.
- **Common traps:** coefficient/scheduler mismatch, prediction-type mismatch, excessive CFG.
- **Interview one-liner:** "Diffusion trains a time-conditioned denoiser on directly sampled noise levels, then numerically reverses noise into data."

20. Final Cheat Sheet

Item	Summary
Definition	Generative model that reverses gradual corruption
Input/output	Noise + condition → data sample
Main steps	Noise data → predict target → iterate reverse scheduler
Key hyperparameters	Schedule, timesteps, target, guidance, sampler, NFE
Metrics	FID/KID, alignment, diversity, latency
Pros	Stable training, quality, coverage, controllability
Cons	Slow iterative inference, compute-intensive
Best use cases	Image/video/audio generation and editing

Stable Diffusion Basics

1. Overview

Stable Diffusion is a family/design pattern of text-conditioned latent diffusion systems. An image VAE compresses pixels into spatial latents, a text encoder converts the prompt to embeddings, and a conditional U-Net or Transformer denoises latent noise using cross-attention. The VAE decoder converts the final latent into an image. This architecture makes high-resolution text-to-image generation feasible on consumer GPUs.

2. Intuition

Rather than repeatedly editing every pixel, Stable Diffusion works on a compressed “visual shorthand.” A text encoder supplies the meaning of the prompt, and cross-attention lets each visual region consult relevant words while noise becomes a latent image. A decoder then expands the shorthand back to pixels.

3. Prerequisites

- Diffusion forward/reverse processes and classifier-free guidance
- Autoencoders/VAEs and spatial latent tensors
- Transformers, tokenization, embeddings, and cross-attention
- U-Nets, schedulers, mixed precision, and GPU memory
- Prompting, negative prompts, seed reproducibility, and image safety

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Image VAE	Compresses/decompresses images	Reduces denoising cost	512px → 64px latent	Lossy compression
Text encoder	Maps prompt tokens to embeddings	Provides semantic condition	CLIP text encoder	Token limit
Latent denoiser	Predicts noise/velocity in latent space	Main generative model	conditional U-Net	Why not pixels
Cross-attention	Visual queries attend to text K/V	Aligns regions with prompt	“red car”	Conditioning path
Scheduler	Defines inference timesteps/update	Speed-quality behavior	DDIM/Euler/DPM	Scheduler ≠ model
CFG scale	Strength of prompt guidance	Alignment-diversity control	7.5	Requires uncond branch
Seed	Initializes latent noise	Enables reproducible comparison	seed 42	Not universal across settings

5. Algorithm / Working Process

1. Tokenize positive and optional negative prompts; run the text encoder.

2. Sample latent noise with shape determined by image resolution and VAE scale.
3. For each scheduler timestep, run the denoiser on latents with unconditional and conditional text embeddings.
4. Combine predictions with CFG and let the scheduler update the latent.
5. Rescale and decode the final latent with the VAE decoder.
6. Convert to an image, run safety/provenance checks, and return seed/settings for reproducibility.

Training encodes real images, adds noise to their latents, and minimizes diffusion prediction loss using captions. Inference uses only the text encoder, iterative denoiser, scheduler, and VAE decoder.

6. Mathematical Foundation

Let $z_0 = sE(x)$ be the scaled VAE latent. Noising and denoising use the standard diffusion relation

$$z_t = \sqrt{\bar{\alpha}_t} z_0 + \sqrt{1 - \bar{\alpha}_t} \epsilon.$$

The denoiser learns

$$\mathcal{L} = E \|\epsilon - \epsilon_\theta(z_t, t, e_{text})\|_2^2$$

or a velocity target. Cross-attention is

$$\text{softmax} \left(\frac{Q_{image} K_{text}^\top}{\sqrt{d}} \right) V_{text}.$$

CFG with a negative/unconditional embedding uses

$$\hat{\epsilon} = \epsilon_{neg} + w(\epsilon_{pos} - \epsilon_{neg}).$$

The VAE scaling constant is checkpoint-specific; omitting it changes the latent distribution seen by the denoiser and produces poor output.

7. Practical Implementation

```
import torch
from diffusers import StableDiffusionPipeline

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32
pipe = StableDiffusionPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", torch_dtype=dtype
).to(device)

generator = torch.Generator(device=device).manual_seed(42)
image = pipe(
    prompt="a clean isometric illustration of a neural network laboratory",
    negative_prompt="blurry, distorted text, watermark",
    num_inference_steps=30,
    guidance_scale=7.0,
    generator=generator,
    height=512,
    width=512,
```

```
).images[0]  
image.save("stable_diffusion_sample.png")
```

8. Code Explanation

The pipeline bundles tokenizer, text encoder, denoiser, scheduler, and VAE. Half precision lowers CUDA memory; CPU retains float32 for compatibility. A local **Generator** makes the seed explicit. Prompt, dimensions, model version, scheduler, precision, and library version all affect reproducibility. The negative prompt supplies the CFG “unconditional” branch; it is not a guaranteed ban list.

9. Training / Evaluation

Caption quality and image-caption alignment are critical. Filter duplicates, watermarks, unsafe content, and low-quality pairs; split by source/identity. Evaluate FID/KID, CLIP or task-specific alignment, aesthetic/human preference, diversity, text rendering, compositional benchmarks, safety, memorization, and inference latency. For adaptation, compare LoRA/DreamBooth checkpoints using fixed seeds and held-out prompts to detect overfitting and identity leakage.

10. Complexity and Cost

VAE compression commonly reduces each spatial dimension by a factor such as 8, giving roughly 64× fewer spatial positions than pixels at equal channel assumptions. Denoising still requires multiple U-Net/Transformer passes, doubled naively for CFG (often batched together). Memory depends on resolution, attention maps, batch size, precision, denoiser size, and text length. Attention slicing, VAE tiling, CPU offload, quantization, and efficient attention trade speed for memory.

11. Common Use Cases

- Text-guided concept art and product visualization
- Inpainting/outpainting and reference-image editing
- LoRA-based style or subject adaptation
- Synthetic images for prototyping and augmentation
- Design tools with ControlNet, masks, and composition controls

12. Common Mistakes

- Forgetting the checkpoint’s VAE latent scaling
- Treating negative prompts as strict logical constraints
- Comparing seeds after changing scheduler/model/resolution
- Using dimensions incompatible with the model’s downsampling factor
- Raising CFG until colors/artifacts degrade
- Fine-tuning on too few repeated images without regularization
- Ignoring checkpoint license, consent, watermarking, and safety requirements

13. Edge Cases / Limitations

Prompt token limits can truncate key phrases. Models may fail at counting, relationships, readable text, uncommon objects, exact brand geometry, and consistent characters across images. The VAE loses fine detail.

Different schedulers produce different results even with the same seed. Model outputs reflect dataset bias and may resemble training samples.

14. Variations

Variation	What changes	When to use	Importance
SD 1.x/2.x	Resolution, text encoder/data/objective differ	Legacy/lightweight projects	Practical
SDXL	Larger base/refiner-style architecture and conditioning	Higher-quality images	Important
LoRA	Low-rank adapter weights	Lightweight style/domain tuning	Essential projects
DreamBooth	Subject-specific fine-tuning	Personal/product identity	Common interview
Inpainting pipeline	Adds masked image condition	Local editing	Essential
Turbo/Lightning/LCM	Distilled/few-step generation	Interactive latency	Systems relevant

15. Related Topics

- **Stable Diffusion vs diffusion:** Stable Diffusion is a text-conditioned latent implementation of the broader diffusion idea.
- **VAE vs denoiser:** VAE handles pixel-latent conversion; the denoiser models the latent distribution.
- **LoRA vs DreamBooth:** LoRA is a parameter-efficient update format; DreamBooth is a subject-learning training method and can use LoRA.
- **CFG vs negative prompt:** negative text defines the comparison branch; CFG scale sets how strongly to move away from it toward positive text.
- **ControlNet vs cross-attention:** cross-attention supplies semantic text; ControlNet supplies spatial control features.

16. Interview Questions

1. **Why is Stable Diffusion called latent diffusion?** The iterative diffusion process runs in a learned compressed image latent space.
2. **What are its main components?** Tokenizer/text encoder, conditional denoiser, scheduler, VAE encoder/decoder, and safety/application logic.
3. **What does cross-attention do?** Image features query text embeddings so spatial synthesis can depend on prompt tokens.
4. **Why is the VAE useful?** It cuts repeated denoising compute while preserving perceptually important image information.
5. **What does guidance scale control?** Strength of conditional direction relative to negative/unconditional prediction.
6. **Why can high CFG hurt?** It over-extrapolates the conditional score, reducing diversity and causing saturation/artifacts.

7. **What does the scheduler do?** Chooses timesteps and numerical update rules; it can trade number of evaluations against quality.
8. **Can the same seed always recreate an image?** Only with the same model, scheduler, settings, dimensions, precision, software, and sufficiently deterministic hardware.
9. **Why use a negative prompt?** It describes features for the CFG baseline to move away from, but does not enforce hard exclusion.
10. **How is LoRA efficient?** It trains low-rank weight updates while freezing most base parameters.
11. **Where does image-to-image strength act?** It determines the starting noise timestep and therefore how much source structure is preserved.
12. **What should production monitoring include?** Latency, failures, unsafe output, prompt/output drift, quality sampling, model/version lineage, and cost.

17. Practice Tasks

- **Coding:** build a reproducible text-to-image script that records all settings.
- **Dataset project:** train a small LoRA on a licensed product/style dataset.
- **Experiment:** sweep steps and CFG, scoring alignment, diversity, and latency.
- **Debugging:** diagnose blank/noisy images caused by dtype or latent scaling errors.
- **Extension:** add inpainting and ControlNet to the same interface.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Brand concept studio	Generates style-controlled mockups	Diffusers, LoRA, Gradio	Licensed brand assets	Adaptation + product UX
Interior editor	Reimagines rooms with masks/control	SDXL, ControlNet, OpenCV	Interior images	Multicondition pipeline
SD benchmark service	Tracks quality, VRAM, latency	FastAPI, MLflow	Fixed prompt suite	MLOps and evaluation

19. Quick Revision

- **Key idea:** text-conditioned diffusion in compressed VAE latents.
- **Main formula:** latent noise prediction plus CFG.
- **When to use:** efficient high-resolution image generation/editing.
- **Metrics:** alignment, FID/KID, human preference, safety, latency/VRAM.
- **Common traps:** scaling/dtype mismatch, excessive CFG, seed reproducibility assumptions.
- **Interview one-liner:** "Stable Diffusion combines a text encoder, cross-attentive latent denoiser, scheduler, and VAE decoder."

20. Final Cheat Sheet

Item	Summary
Definition	Text-conditioned latent diffusion architecture
Input/output	Prompt + seed/controls → image

Item	Summary
Main steps	Encode text → denoise latent → VAE decode
Key hyperparameters	Steps, scheduler, CFG, seed, resolution, strength
Metrics	FID/KID, CLIP/alignment, preference, latency, safety
Pros	High quality, flexible ecosystem, consumer-GPU feasible
Cons	Iterative latency, VAE loss, prompt/composition failures
Best use cases	Text-to-image, editing, inpainting, personalized styles

Conditional Generation

1. Overview

Conditional generation models $p(x \mid c)$: it creates an output x while obeying condition c , such as a class, text prompt, source image, segmentation map, pose, depth map, audio, or multiple controls. Conditioning converts an unconstrained generator into a useful controllable system and appears in conditional VAEs/GANs, encoder-decoder LMs, diffusion models, translation, and structured prediction.

2. Intuition

Unconditional generation asks a chef to “make anything.” Conditional generation supplies an order: dish, ingredients, allergies, and presentation. The generator still decides details, but the condition narrows the valid output distribution.

3. Prerequisites

- Conditional probability and Bayes’ rule
- Embeddings, concatenation, normalization, and attention
- VAEs, GANs, autoregressive models, or diffusion basics
- Paired/labeled datasets and missing-condition handling
- Evaluation of both quality and condition compliance

4. Core Concepts

Concept	Meaning	Why it matters	Simple example	Interview angle
Condition c	Information output should respect	Provides control	label “cat”	Discrete vs continuous
Early fusion	Concatenate condition near input	Simple/global control	append class embedding to z	Spatial mismatch

Concept	Meaning	Why it matters	Simple example	Interview angle
Conditional normalization	Modulate scale/bias using c	Strong layer-wise influence	AdaIN/FiLM	Style control
Cross-attention	Queries attend to condition sequence	Flexible token-level control	text-to-image	Q from output features
Spatial conditioning	Condition aligned with output coordinates	Preserves layout	edge/depth map	Concatenate/control branch
Condition dropout	Randomly remove c in training	Enables CFG/robustness	empty prompt	Drop rate trade-off
Multicondition fusion	Combines text, image, pose, etc.	Real applications need several controls	prompt + depth	Conflicting signals

5. Algorithm / Working Process

1. Encode condition into a vector, token sequence, or spatial feature map.
2. Encode/supply stochastic source z or noisy state x_t .
3. Fuse condition using concatenation, FiLM/normalization, cross-attention, or a control branch.
4. Train against the target output using the family's loss while checking that matched pairs are correct.
5. Optionally include condition dropout, mismatch negatives, or auxiliary condition classifiers.
6. At inference, provide condition and generation controls; validate both output quality and adherence.

6. Mathematical Foundation

Conditional maximum likelihood minimizes

$$\mathcal{L} = -\mathbb{E}_{(x,c) \sim p_{data}} \log p_{\theta}(x | c).$$

An autoregressive conditional model factorizes

$$p(x_{1:T} | c) = \prod_t p(x_t | x_{<t}, c).$$

A conditional GAN plays

$$\min_G \max_D E_{x,c} \log D(x, c) + E_{z,c} \log(1 - D(G(z, c), c)).$$

FiLM transforms a feature h as $h' = \gamma(c) \odot h + \beta(c)$. Conditional diffusion trains $E \|\epsilon - \epsilon_{\theta}(x_t, t, c)\|^2$ and may apply classifier-free guidance. Mutual information $I(X; C)$ is a useful conceptual measure of whether outputs actually depend on the condition.

7. Practical Implementation

```

import torch
from torch import nn

class ConditionalGenerator(nn.Module):
    def __init__(self, classes=10, z_dim=64, emb_dim=16):
        super().__init__()
        self.label_embedding = nn.Embedding(classes, emb_dim)
        self.net = nn.Sequential(
            nn.Linear(z_dim + emb_dim, 256), nn.ReLU(),
            nn.Linear(256, 28 * 28), nn.Sigmoid()
        )

    def forward(self, noise, labels):
        condition = self.label_embedding(labels)
        return self.net(torch.cat([noise, condition], dim=1)).view(-1, 1, 28, 28)

model = ConditionalGenerator()
noise = torch.randn(8, 64)
labels = torch.tensor([0, 1, 2, 3, 4, 5, 6, 7])
images = model(noise, labels)
assert images.shape == (8, 1, 28, 28)

```

8. Code Explanation

Embedding learns a dense representation for each class. Concatenating it with random noise is early fusion: noise controls within-class variation while the label controls class identity. A complete cGAN also passes the same labels to the discriminator; otherwise the discriminator can reward realism without enforcing label consistency.

9. Training / Evaluation

Ensure conditions and targets stay aligned through augmentation. Split by subject/source to prevent leakage. Measure base quality plus condition-specific metrics: classifier accuracy for labels, CLIP score for text, keypoint/depth/segmentation error for spatial controls, semantic consistency, diversity within each condition, and human ratings. Audit class balance and rare condition combinations. Tune condition-drop rate, embedding size, guidance, fusion depth, and condition weights.

10. Complexity and Cost

Simple vector concatenation adds little cost. Cross-attention costs roughly $O(N_{out}N_{cond})$ and stores attention activations. Spatial control networks may add nearly another encoder/U-Net path. Classifier-free guidance commonly requires unconditional and conditional predictions, increasing inference compute, though they can share a batch.

11. Common Use Cases

- Label-guided image generation and balanced augmentation
- Translation, summarization, and question answering
- Text-to-image/audio/video generation

- Pose-, edge-, depth-, or segmentation-guided image creation
- Personalized generation from identity/style references

12. Common Mistakes

- Conditioning the generator but not the GAN discriminator
- Breaking input-condition alignment during data augmentation
- Evaluating realism but not adherence
- Using global vector conditions for precise spatial constraints
- Training CFG without unconditional condition dropout
- Allowing one strong condition to overwhelm all others
- Testing only frequent labels or easy prompt combinations

13. Edge Cases / Limitations

Conditions may conflict, be ambiguous, missing, noisy, or outside the training support. Models may ignore weak conditions or overfit spurious correlations—for example, class labels tied to background. Strong conditioning can reduce diversity. Cross-attention does not guarantee binding (“red cube beside blue sphere”), exact counts, or hard constraints.

14. Variations

Variation	What changes	When to use	Importance
Class conditional	Embeds discrete label	Category control	Basic
Text conditional	Uses text token embeddings	Open-vocabulary control	Essential
Spatial conditional	Uses map aligned to output	Layout/pose/edges	Project essential
Conditional normalization	Condition predicts feature scale/bias	Style/global attributes	Important
Classifier guidance	Uses gradient of external classifier	Retrofitting diffusion control	Advanced
Classifier-free guidance	Joint conditional/unconditional model	Strong modern alignment	Essential

15. Related Topics

- **Conditional vs unconditional generation:** conditional narrows the modeled distribution using side information.
- **Concatenation vs cross-attention:** concatenation suits fixed vectors; cross-attention suits variable-length token conditions.
- **Soft vs hard constraints:** conditioning encourages behavior; constrained decoding/optimization can enforce validity.
- **CFG vs ControlNet:** CFG strengthens semantic condition; ControlNet introduces an explicit spatial feature pathway.
- **Conditioning vs retrieval:** retrieval supplies external evidence that becomes another condition.

16. Interview Questions

1. **What is conditional generation?** Modeling and sampling $p(x \mid c)$ rather than the marginal $p(x)$.
2. **How can a condition enter a network?** Concatenation, additive embeddings, FiLM/conditional normalization, cross-attention, or spatial control branches.
3. **Why condition the discriminator?** Otherwise it checks only realism, not whether fake output matches the requested condition.
4. **What is classifier-free guidance training?** Randomly replace some conditions with null so one model learns both conditional and unconditional predictions.
5. **How do you measure adherence?** Use a condition-specific recognizer/metric plus human evaluation and adversarial edge-case tests.
6. **Why can the model ignore c ?** Output can minimize the main loss using correlations or a dominant autoregressive/history path without relying on condition.
7. **How can condition usage be strengthened?** Better fusion, auxiliary losses, mismatched negatives, balanced data, attention/control pathways, or guidance.
8. **Why are spatial maps different from labels?** They specify location-dependent constraints that a single global embedding may lose.
9. **What if conditions conflict?** Define priority/weights, train on such combinations, detect conflicts, or ask users to resolve them.
10. **What is compositional generalization?** Following novel combinations of familiar conditions not observed together in training.
11. **Does high guidance equal hard control?** No; it biases probabilities but does not mathematically enforce the condition.
12. **How should missing conditions be handled?** Train with explicit null/missing representations and define safe fallback behavior.

17. Practice Tasks

- **Coding:** build a class-conditional MNIST generator.
- **Dataset project:** translate captions into small images or icons.
- **Experiment:** compare concatenation, FiLM, and cross-attention.
- **Debugging:** diagnose a cGAN that generates realistic but wrong labels.
- **Extension:** combine class and spatial mask conditions with conflict tests.

18. Project Ideas

Project	What it does	Tech stack	Dataset	Resume value
Pose-guided avatar	Generates character under a skeleton pose	PyTorch, ControlNet	DeepFashion/pose pairs	Spatial conditioning
Conditional synthetic lab	Balances rare classes	cGAN/diffusion, sklearn	Imbalanced CV dataset	Measures downstream utility
Controlled text generator	Generates schema-valid domain text	Transformers, FastAPI	Public domain corpus	Constraints + evaluation

19. Quick Revision

- **Key idea:** generate from $p(x \mid c)$ by injecting useful side information.
- **Main formula:** conditional NLL $-E \log p_\theta(x \mid c)$.
- **When to use:** whenever users need semantic, structural, or reference control.
- **Metrics:** base quality plus adherence and within-condition diversity.
- **Common traps:** misaligned pairs, ignored conditions, confusing guidance with guarantees.
- **Interview one-liner:** "A good conditional generator must be both realistic and dependent on the condition; evaluate both."

20. Final Cheat Sheet

Item	Summary
Definition	Generation governed by auxiliary information c
Input/output	Noise/source + condition $\rightarrow$ matching output
Main steps	Encode condition $\rightarrow$ fuse $\rightarrow$ generate $\rightarrow$ assess adherence
Key hyperparameters	Embedding/fusion, condition drop, guidance, control weights
Metrics	Quality, condition accuracy/alignment, diversity
Pros	Useful control, personalization, task-specific output
Cons	Condition ignoring, conflicts, biased correlations
Best use cases	Text/label/spatial/reference-guided generation

Text-to-Image Models

1. Overview

Text-to-image (T2I) models generate images conditioned on natural-language prompts. Current systems usually combine a pretrained text encoder with a latent diffusion U-Net or diffusion Transformer, though autoregressive image-token models also exist. T2I powers illustration, design ideation, advertisements, storyboards, synthetic data, and visual prototyping.

2. Intuition

The prompt is a semantic blueprint, not a pixel-perfect specification. The text encoder represents words and relationships; the image generator uses those representations to choose content while random noise controls variation. Reusing a seed keeps the initial randomness fixed for controlled comparisons.

3. Prerequisites

- Tokenization, text embeddings, Transformers, and CLIP
- Conditional generation, diffusion, and CFG
- CNN/U-Net or diffusion Transformer architecture

- Captioned datasets and image preprocessing
- Generative quality, alignment, safety, and bias evaluation

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Prompt embedding	Token-level semantic representation	Conditions synthesis	"red robot"	Text encoder may be frozen
Cross-attention	Image queries attend to prompt tokens	Connects spatial features to language	region attends to "robot"	Attention is not proof of binding
Caption quality	Accuracy/detail of training text	Upper-bounds controllability	alt text vs rich caption	Data curation
CFG	Strengthens prompt-conditioned direction	Improves alignment	scale 5–8	Diversity trade-off
Negative prompt	Reference branch describing undesired traits	Heuristic steering	"blurry"	Not hard exclusion
Compositionality	Combines attributes/objects/relations	Key usability challenge	red cube left of blue ball	Binding/counting tests
Prompt expansion	Rewrites short input descriptively	Improves aesthetics/detail	add lighting/style	Can alter intent

5. Algorithm / Working Process

Training pairs an image x with caption c : encode c , corrupt image/latent at random t , predict the diffusion target conditioned through cross-attention, and optimize denoising loss. Inference tokenizes the prompt, samples noise, iteratively performs CFG-guided denoising, decodes the final latent, and runs safety/provenance processing.

6. Mathematical Foundation

The goal is $p_\theta(x \mid c)$. Latent diffusion commonly minimizes

$$E_{x,c,t,\epsilon} \|\epsilon - \epsilon_\theta(z_t, t, E_{text}(c))\|_2^2.$$

Cross-attention uses Q from image features and K, V from text. CFG estimates a sharper conditional score using $\epsilon_u + w(\epsilon_c - \epsilon_u)$. CLIP alignment is often approximated by cosine similarity

$$s(I, T) = \frac{f_I(I)^\top f_T(T)}{\|f_I(I)\| \|f_T(T)\|},$$

but high CLIP score alone does not ensure visual quality or correct relationships.

7. Practical Implementation

```
import torch
from diffusers import AutoPipelineForText2Image

device = "cuda" if torch.cuda.is_available() else "cpu"
pipe = AutoPipelineForText2Image.from_pretrained(
    "stabilityai/sdxl-turbo",
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
).to(device)

prompt = "editorial photo of a solar-powered library bus in rural India, morning light"
g = torch.Generator(device=device).manual_seed(123)
image = pipe(prompt, num_inference_steps=4, guidance_scale=0.0,
generator=g).images[0]
image.save("text_to_image.png")
```

8. Code Explanation

`AutoPipelineForText2Image` selects components matching the checkpoint. This distilled checkpoint is designed for few steps and its recommended low/zero CFG behavior; settings should not be copied blindly to ordinary SD/SDXL checkpoints. The explicit generator controls initialization. Production code should log model revision, prompt, seed, scheduler, dimensions, and content-policy result.

9. Training / Evaluation

Train on licensed, deduplicated, correctly captioned pairs; split by source/artist/identity. Evaluate FID/KID, CLIP or retrieval alignment, GenEval-style object/count/relation tests, OCR accuracy, diversity, human preference, safety, bias, and memorization. Prompt suites must cover short/long prompts, rare concepts, negation, relations, and demographic parity. Tune caption dropout, resolution buckets, SNR weighting, CFG, steps, and sampler.

10. Complexity and Cost

Text encoding is usually small compared with repeated image denoising. Compute rises roughly with latent token count and number of function evaluations; attention cost grows strongly with resolution. Training requires large paired corpora and accelerators. Distillation/few-step samplers reduce latency; caching prompt embeddings helps repeated prompts.

11. Common Use Cases

Concept art, marketing drafts, storyboards, product visualization, education, game assets, synthetic data, and personalized illustration.

12. Common Mistakes

- Treating prompts as deterministic programs
- Using one aesthetic score as the whole evaluation
- Ignoring caption noise and duplicate leakage

- Copying CFG/step settings across incompatible checkpoints
- Claiming negative prompts guarantee exclusion
- Fine-tuning without consent/license checks

13. Edge Cases / Limitations

Exact text, counting, left/right relations, uncommon cultural concepts, and consistent identities are difficult. Prompt injection can matter when prompts come from untrusted sources in a larger pipeline. Bias and memorization reflect training data. Generation cannot establish legal ownership or factual authenticity.

14. Variations

Variation	Change	When to use	Importance
Pixel diffusion	Denoises pixels	Maximum fidelity research	Advanced
Latent diffusion	Denoises VAE latents	Practical high resolution	Essential
Autoregressive T2I	Predicts image tokens	Unified sequence modeling	Research
Cascaded model	Base then super-resolution	Very high resolution	Important
Personalized T2I	LoRA/DreamBooth/reference adapters	Subject/style consistency	Projects

15. Related Topics

T2I vs image-to-image: T2I starts from noise; image-to-image also conditions on source structure. **CLIP vs generator:** CLIP aligns modalities but does not itself decode pixels. **Cross-attention vs ControlNet:** semantic token conditioning versus spatial control. **Prompting vs LoRA:** transient instruction versus learned specialization.

16. Interview Questions

1. **What distribution is learned?** $p(\text{image} \mid \text{text})$.
2. **Why use cross-attention?** It lets spatial image features selectively use variable-length text tokens.
3. **Why are captions crucial?** Noisy or incomplete captions teach weak or incorrect text-image associations.
4. **What controls diversity?** Seed/noise, sampler stochasticity, guidance, and model/data coverage.
5. **Why does CFG help?** It extrapolates from unconditional toward conditional prediction, strengthening prompt relevance.
6. **Why can CFG hurt?** Excessive extrapolation reduces diversity and causes artifacts/saturation.
7. **How test compositionality?** Controlled prompts varying object, attribute, count, and spatial relation with detectors plus human review.
8. **Why is CLIP score insufficient?** It may reward broad semantic match while missing geometry, quality, count, or bias.
9. **What is prompt truncation?** Tokens beyond text encoder length are ignored or require special chunking.
10. **How prevent memorization?** Deduplicate, regularize, audit nearest neighbors, restrict data, and monitor privacy.

11. **Why do distilled models need different settings?** Their training targets a particular short sampler/guidance regime.
12. **What metadata enables reproducibility?** Checkpoint/revision, scheduler, prompt, negative prompt, seed, size, steps, guidance, precision, software.

17. Practice Tasks

- **Small coding task:** implement a prompt/seed grid.
- **Dataset project:** benchmark a licensed caption-image subset.
- **Experiment:** sweep CFG and steps on relation prompts.
- **Debugging:** diagnose prompt truncation and seed mismatch.
- **Extension:** train and evaluate a licensed LoRA.

18. Project Ideas

Project	What it does	Stack	Dataset	Resume value
Prompt benchmark	Tests alignment/composition	Diffusers, CLIP, Gradio	Curated prompts	Evaluation depth
Catalog concept tool	Creates product backgrounds	SDXL, FastAPI	Licensed catalog	Product pipeline
Inclusive data audit	Measures demographic bias	Python, detectors	Balanced prompt set	Responsible AI

19. Quick Revision

- **Key idea:** language embeddings condition a visual generator.
- **Main formula:** conditional denoising loss.
- **Metrics:** FID/KID, alignment, composition, preference, safety.
- **Trap:** semantic similarity is not exact prompt compliance.
- **One-liner:** "T2I maps text into conditioning features and samples the conditional image distribution."

20. Final Cheat Sheet

Item	Summary
Definition	Conditional generation of images from language
Input/output	Prompt + noise → image
Main steps	Encode text → denoise/generate → decode → validate
Key hyperparameters	Seed, steps, CFG, sampler, resolution
Metrics	FID/KID, alignment, composition, preference, safety
Pros	Open-vocabulary creativity and rapid visual ideation
Cons	Weak exact control, bias, memorization risk, high cost
Best uses	Ideation, assets, synthetic imagery

Image-to-Image Models

1. Overview

Image-to-image (I2I) models transform a source image into a target image while preserving selected content. Tasks include colorization, super-resolution, style transfer, restoration, segmentation-to-photo, day-to-night conversion, and prompt-guided editing. Training may use paired data, unpaired data, or a pretrained diffusion model.

2. Intuition

The source image is a draft: the model keeps its layout but redraws appearance according to a target domain or text instruction. A “strength” control decides whether the output is a light retouch or a major reinterpretation.

3. Prerequisites

CNNs/U-Nets, GANs and diffusion, perceptual losses, aligned vs unpaired datasets, image registration/augmentation, and structural evaluation.

4. Core Concepts

Concept	Meaning	Why it matters	Example	Interview angle
Paired mapping	Aligned (x, y) examples	Direct supervised target	sketch/photo pair	Pix2Pix
Unpaired mapping	Separate source/target sets	Easier data collection	horses/zebras	Cycle consistency
Content preservation	Retain source structure	Defines edit usefulness	same pose	Identity/perceptual loss
Style/domain transfer	Change visual distribution	Main transformation	summer→winter	Content-style trade-off
Diffusion strength	Noise applied to source latent	Controls edit magnitude	0.2 subtle, 0.8 large	Starting timestep
Masking	Restricts editable region	Local edits	replace background	Inpainting

5. Algorithm / Working Process

Paired training encodes source, predicts target, and uses reconstruction plus adversarial/perceptual loss. Unpaired translation trains mappings in both directions and enforces cycle/identity consistency. Diffusion I2I encodes the source to z_0 , adds noise at a strength-selected timestep, then denoises under text/control conditions and decodes.

6. Mathematical Foundation

Paired objective: $\mathcal{L} = \mathcal{L}_{adv} + \lambda \|y - G(x)\|_1$. Cycle consistency uses

$$\mathcal{L}_{cyc} = E_x \|F(G(x)) - x\|_1 + E_y \|G(F(y)) - y\|_1.$$

Diffusion editing initializes $z_t = \sqrt{\bar{\alpha}_t}E(x) + \sqrt{1 - \bar{\alpha}_t}\epsilon$. Larger t removes more source information. Perceptual loss compares $\phi_l(y)$ and $\phi_l(\hat{y})$ rather than only pixels.

7. Practical Implementation

```
import torch
from PIL import Image
from diffusers import AutoPipelineForImage2Image

device = "cuda" if torch.cuda.is_available() else "cpu"
pipe = AutoPipelineForImage2Image.from_pretrained(
    "runwayml/stable-diffusion-v1-5",
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
).to(device)
source = Image.open("source.png").convert("RGB").resize((512, 512))
result = pipe(
    prompt="watercolor architectural illustration, preserve building geometry",
    image=source, strength=0.45, guidance_scale=7.0, num_inference_steps=30,
).images[0]
result.save("edited.png")
```

8. Code Explanation

The pipeline VAE-encodes the source, corrupts its latent according to `strength`, and denoises toward the prompt. Low strength preserves more source pixels; high strength enables larger changes. Resize/crop policy matters because it can change composition before generation.

9. Training / Evaluation

Use aligned splits for paired tasks and identity/source splits for people/products. Metrics depend on goal: PSNR/SSIM for fidelity, LPIPS for perception, FID for target realism, segmentation/keypoint consistency for structure, OCR/identity scores where relevant, and human preference. Always report source-preservation and target-domain success separately.

10. Complexity and Cost

One-pass CNN/GAN translation is fast. Diffusion editing costs multiple denoiser evaluations plus VAE encoding/decoding. High-resolution perceptual losses consume memory. Paired dataset creation and alignment can dominate engineering cost.

11. Common Use Cases

Restoration, colorization, super-resolution, virtual try-on, medical modality translation, style transfer, background replacement, and design rendering.

12. Common Mistakes

Misaligned pairs; identical augmentation not applied to source/target; random frame leakage; reporting only realism; strength too high; using unpaired translation where exact semantic preservation is safety-critical.

13. Edge Cases / Limitations

Unpaired models may change medically or semantically important content because cycle consistency does not guarantee truthful mapping. Diffusion edits can alter identity, text, small objects, or geometry. Large domain gaps and rare source structures fail more often.

14. Variations

Variation	Change	Use	Importance
Pix2Pix	Paired cGAN + L1	Aligned translation	Essential
CycleGAN	Unpaired + cycle loss	Unpaired domains	Essential
Neural style transfer	Content/style feature losses	Artistic transfer	Foundational
Diffusion I2I	Noisy source latent + prompt	Flexible editing	Modern essential
Inpainting	Masked local generation	Object edit/removal	Project essential

15. Related Topics

I2I vs T2I: source image adds structural information. **Pix2Pix vs CycleGAN:** paired supervision versus unpaired consistency. **Restoration vs generation:** restoration should preserve ground truth; generative priors may invent details. **ControlNet vs I2I:** ControlNet supplies an explicit extracted structure rather than relying only on noisy source latent.

16. Interview Questions

- Paired vs unpaired I2I?** Paired uses aligned targets; unpaired learns domain mappings without one-to-one examples.
- Why L1 over L2 in Pix2Pix?** L1 often preserves edges with less averaging blur.
- What is cycle consistency?** Translating there and back should reconstruct the source.
- Does cycle consistency ensure semantic correctness?** No; mappings can hide information or change critical content while remaining invertible.
- What does diffusion strength mean?** The source-latent noise level/starting timestep, controlling retained information.
- How evaluate style transfer?** Separate content preservation, style/domain match, artifacts, diversity, and human preference.
- Why use perceptual loss?** Feature-space distance correlates better with semantic/perceptual similarity than exact pixels.
- How prevent pair misalignment?** Joint deterministic geometry transforms and registration checks.

9. **When is hallucinated detail dangerous?** Medical, forensic, scientific, and restoration settings where output is treated as evidence.
10. **Why use identity loss in CycleGAN?** It discourages unnecessary change when input is already in the target domain.
11. **What is inpainting?** Generate only a masked region while conditioning on surrounding pixels and optional prompt.
12. **How preserve faces/products?** Lower strength, identity/reference losses or adapters, masks, structural controls, and explicit evaluation.

17. Practice Tasks

- **Small coding task:** train a minimal Pix2Pix mapping.
- **Dataset project:** translate CMP facade labels to photos.
- **Experiment:** compare PSNR with LPIPS across diffusion strengths.
- **Debugging:** repair source-target augmentation misalignment.
- **Extension:** add mask-restricted editing.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
Scan restorer	Cleans documents	U-Net/Pix2Pix, OpenCV	Paired synthetic noise	Practical CV
Facade renderer	Label map → building	Pix2Pix	CMP Facades	Paired generation
Safe photo editor	Masked prompt edits	Diffusers, FastAPI	User/licensed images	Product deployment

19. Quick Revision

- **Key idea:** transform appearance while preserving selected source content.
- **Formula:** adversarial + reconstruction, cycle, or noisy-source diffusion.
- **Metrics:** preservation and target realism separately.
- **Trap:** plausible output is not necessarily faithful.
- **One-liner:** "I2I learns $p(y \mid x, c)$, with supervision and losses defining what must stay unchanged."

20. Final Cheat Sheet

Item	Summary
Definition	Conditional transformation of a source image
Input/output	Source image + optional prompt/mask → transformed image
Main steps	Encode → transform/denoise → decode → verify preservation
Key hyperparameters	Strength, loss weights, CFG, mask, steps
Metrics	PSNR/SSIM, LPIPS, FID, structure/identity consistency

Item	Summary
Pros	Strong structural starting point and local editing
Cons	Can hallucinate, drift, or corrupt important source detail
Best uses	Editing, translation, restoration, rendering

ControlNet Idea

1. Overview

ControlNet adds spatial conditioning to a pretrained text-to-image diffusion model using a trainable control branch connected to a mostly frozen base network. It accepts edge maps, poses, depth, segmentation, line art, normals, or other aligned controls. The key idea is to preserve pretrained image quality while learning how spatial signals should steer denoising.

2. Intuition

Text says what to draw; ControlNet supplies tracing paper showing where it should go. A copied network branch learns to translate the tracing into adjustments, initially connected through zero-output layers so the pretrained model's behavior is not damaged at the start.

3. Prerequisites

Stable Diffusion, U-Net feature hierarchy, residual/skip connections, spatial maps, paired data, frozen vs trainable weights, and classifier-free guidance.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Control image	Spatial condition aligned to output	Precise geometry	Canny edges	Preprocessor quality
Locked copy	Frozen pretrained U-Net blocks	Retains base capability	original backbone	Stable fine-tuning
Trainable copy	Learns control features	Adapts new condition	cloned encoder blocks	Added compute
Zero convolution	Zero-initialized 1×1 connection	Starts as no-op	residual injection	Why stable initialization
Control scale	Strength per branch/timestep	Balances prompt and structure	pose weight 0.8	Too high causes rigidity
Multi-ControlNet	Several controls	Rich composition	depth + pose	Conflict/compute

5. Algorithm / Working Process

Preprocess a source into control map c_s . Feed noisy latent and c_s through trainable control blocks. Inject their zero-convolution residuals into corresponding frozen denoiser blocks while text enters through cross-attention. Train only control parameters (and any explicitly unfrozen parts) on (image, caption, control) triples. At inference, tune control scale and optionally its active timestep range.

6. Mathematical Foundation

If frozen block output is $F(x; \theta)$ and trainable control branch is $C(x, c; \phi)$, a simplified residual is

$$y = F(x; \theta) + Z(C(x, c; \phi); \psi),$$

where zero convolution $Z(\cdot; \psi)$ has zero-initialized weights/bias, so initially $y = F(x; \theta)$. Training still minimizes diffusion target error $E\|\epsilon - \epsilon_\theta(z_t, t, \text{text}, c_s)\|^2$.

7. Practical Implementation

```
import torch
from PIL import Image
from diffusers import ControlNetModel, StableDiffusionControlNetPipeline

device = "cuda" if torch.cuda.is_available() else "cpu"
dtype = torch.float16 if device == "cuda" else torch.float32
controlnet = ControlNetModel.from_pretrained("lllyasviel/sd-controlnet-canny",
torch_dtype=dtype)
pipe = StableDiffusionControlNetPipeline.from_pretrained(
    "runwayml/stable-diffusion-v1-5", controlnet=controlnet, torch_dtype=dtype
).to(device)
edges = Image.open("canny_edges.png").convert("RGB")
image = pipe("a futuristic railway station, cinematic", image=edges,
             controlnet_conditioning_scale=0.8, num_inference_steps=30).images[0]
image.save("controlled.png")
```

8. Code Explanation

The ControlNet and base checkpoint must be architecturally compatible. The control image is already preprocessed; real systems must produce it at the expected channel format and resolution.

`controlnet_conditioning_scale` trades structural adherence against generative freedom.

9. Training / Evaluation

Build accurately aligned image-caption-control triples and split by source. Evaluate base image quality, text alignment, and control error: edge overlap, pose keypoint error, depth correlation, or segmentation IoU. Test imperfect control maps. Tune control strength, active timestep interval, conditioning resolution, LR, and caption dropout.

10. Complexity and Cost

ControlNet adds a substantial trainable branch and activations; inference adds control-network computation at each denoising step. Multiple controls multiply cost. Frozen base weights reduce optimization state during

training but not all forward compute.

11. Common Use Cases

Pose-guided people, architecture from sketches, depth-preserving restyling, segmentation-to-scene generation, product layout, line-art coloring, and consistent camera composition.

12. Common Mistakes

Wrong preprocessor/checkpoint pair; misaligned resolution; excessive control strength; assuming edges specify object semantics; training on controls extracted after augmentations that no longer align; evaluating only appearance.

13. Edge Cases / Limitations

Ambiguous/noisy controls can produce artifacts. Edge maps lack depth/semantics; depth lacks texture; pose lacks identity. Text and control may conflict. Exact pixel compliance is not guaranteed, and multiple branches consume considerable VRAM.

14. Variations

Variation	Change	Use	Importance
Canny/lineart	Edge control	Shape/layout	Project common
OpenPose	Skeleton control	Human pose	Project common
Depth/normal	Geometry control	Scene consistency	Important
Segmentation	Semantic region map	Layout planning	Important
T2I-Adapter	Lighter adapter pathway	Efficient control	Advanced comparison
IP-Adapter	Image-embedding condition	Style/identity reference	Modern project

15. Related Topics

ControlNet vs CFG: spatial adherence versus text-conditioning strength. **ControlNet vs LoRA:** control interface versus domain/style parameter adaptation. **ControlNet vs image-to-image:** explicit extracted geometry versus retaining a noisy source latent. **ControlNet vs IP-Adapter:** spatial map versus global/patch reference embedding.

16. Interview Questions

1. **What problem does ControlNet solve?** Precise spatial control of a pretrained diffusion generator.
2. **Why freeze the base?** Preserve pretrained generative capability and reduce catastrophic forgetting.
3. **Why zero convolutions?** Control residuals begin at zero, so training starts from the base model's behavior.
4. **What is actually trained?** Control branch and zero connections, plus any intentionally unfrozen modules.

5. **Why not just concatenate an edge map?** A multi-scale branch can inject aligned control throughout the U-Net without retraining the full base.
6. **How evaluate pose ControlNet?** Keypoint detection error/coverage plus image quality and prompt alignment.
7. **What happens with high control scale?** Rigid tracing and artifacts; prompt/style freedom decreases.
8. **Can controls conflict?** Yes; text, pose, depth, and edge constraints may describe incompatible scenes.
9. **Why must control resolution align?** Spatial features are injected at matching U-Net scales.
10. **Does a Canny map contain semantics?** No; text/base priors decide what the edges represent.
11. **What is Multi-ControlNet?** Combining residuals from multiple independently conditioned control branches.
12. **When prefer an adapter?** When lower parameter/compute overhead or a different conditioning interface is more important.

17. Practice Tasks

- **Small coding task:** run Canny-conditioned generation.
- **Dataset project:** evaluate pose control on a fashion subset.
- **Experiment:** sweep control scale and active timestep range.
- **Debugging:** diagnose a wrong preprocessor or misaligned map.
- **Extension:** combine depth and pose and analyze conflicts.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
Sketch renderer	Sketch → product visualization	ControlNet, SDXL	Sketch/photo pairs	Spatial GenAI
Pose studio	Pose-controlled avatars	OpenPose, Diffusers	Fashion images	Multistage CV
Interior planner	Segmentation/depth-guided rooms	ControlNet, Gradio	ADE20K/interiors	User control UX

19. Quick Revision

- **Key:** zero-initialized residual control branch around a frozen diffusion model.
- **Formula:** base feature + scaled learned control residual.
- **Metrics:** control error + quality + text alignment.
- **Trap:** spatial control does not supply semantics or hard guarantees.
- **One-liner:** "ControlNet preserves a pretrained denoiser and learns multi-scale residuals from aligned control maps."

20. Final Cheat Sheet

Item	Summary
Definition	Spatial control branch for a pretrained diffusion model

Item	Summary
Input/output	Prompt + spatial control + noise → controlled image
Main steps	Preprocess → control branch → residual injection → denoise
Key hyperparameters	Control scale/start/end, CFG, steps, preprocessor
Metrics	Control error, FID/KID, text alignment, human preference
Pros	Strong multi-scale layout and geometry control
Cons	Extra compute, imperfect maps, and conflicting conditions
Best uses	Pose, depth, edge, segmentation-guided synthesis

CLIP

1. Overview

CLIP (Contrastive Language–Image Pretraining) learns aligned image and text embeddings from image-caption pairs. Separate encoders map both modalities into one normalized space where matching pairs have high similarity. CLIP supports zero-shot classification, cross-modal retrieval, text-to-image evaluation/conditioning, semantic search, safety classifiers, and multimodal representations; it is not itself an image generator.

2. Intuition

CLIP learns a shared map: a dog photo and “a photo of a dog” land near each other, while unrelated captions land far away. Classification becomes retrieval—ask which class prompt is closest to the image.

3. Prerequisites

CNNs/Vision Transformers, text Transformers, embeddings and cosine similarity, contrastive learning, cross-entropy, batching, and retrieval metrics.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Dual encoder	Separate image/text networks	Efficient independent indexing	ViT + Transformer	No deep cross-attention
Shared space	Projected normalized embeddings	Direct comparison	cosine similarity	L2 normalization
Positive pair	Matched image-caption	Attraction signal	dog + dog caption	Noisy web pairs

Concept	Meaning	Why	Example	Interview angle
In-batch negatives	Other batch items treated as negatives	Scales contrastive learning	N^2 similarities	False negatives
Temperature	Scales similarity logits	Controls softmax sharpness	learned logit scale	Stability/clamping
Prompt template	Text phrasing used for labels	Changes zero-shot accuracy	"a photo of a {}"	Prompt ensembling

5. Algorithm / Working Process

Encode a batch of N images and captions, project and L2-normalize embeddings, calculate the $N \times N$ similarity matrix, and apply cross-entropy in both image-to-text and text-to-image directions with diagonal pairs as targets. At inference, precompute one modality and retrieve by nearest similarity, or compare an image against class-prompt embeddings.

6. Mathematical Foundation

With normalized v_i, t_j , logits are $s_{ij} = v_i^T t_j / \tau$. Symmetric InfoNCE loss is

$$\mathcal{L} = \frac{1}{2} \left[-\frac{1}{N} \sum_i \log \frac{e^{s_{ii}}}{\sum_j e^{s_{ij}}} - \frac{1}{N} \sum_j \log \frac{e^{s_{jj}}}{\sum_i e^{s_{ij}}} \right].$$

Cosine similarity equals dot product after normalization. Zero-shot class probability applies softmax over similarities to class prompt embeddings.

7. Practical Implementation

```
import torch
from PIL import Image
from transformers import CLIPModel, CLIPProcessor

name = "openai/clip-vit-base-patch32"
model = CLIPModel.from_pretrained(name)
processor = CLIPProcessor.from_pretrained(name)
image = Image.open("candidate.jpg").convert("RGB")
labels = ["a photo of a cat", "a photo of a dog", "a photo of a bicycle"]
inputs = processor(text=labels, images=image, return_tensors="pt", padding=True)
with torch.inference_mode():
    probabilities = model(**inputs).logits_per_image.softmax(dim=1)[0]
print(dict(zip(labels, probabilities.tolist())))
```

8. Code Explanation

The processor applies the checkpoint's exact image normalization and text tokenization. `logits_per_image` contains learned-scale similarities. Softmax makes probabilities relative only to the supplied candidate labels; a high value does not mean open-world certainty.

9. Training / Evaluation

Use diverse, deduplicated image-text pairs and source-held-out splits. Evaluate image-to-text/text-to-image Recall@K, median rank, zero-shot accuracy, calibration, subgroup performance, robustness, and downstream retrieval. Large batches improve negatives but increase false-negative risk. Tune encoders, projection size, temperature, batch size, augmentations, and hard-negative strategy.

10. Complexity and Cost

Encoding cost is one pass per item; similarity between N images and M texts is $O(NMd)$ naively but supports matrix multiplication and approximate nearest-neighbor indexing. Training holds two encoders and a batch similarity matrix. Dual encoders make retrieval much cheaper than pairwise cross-encoders.

11. Common Use Cases

Zero-shot classification, semantic image search, content moderation signals, dataset filtering/caption ranking, T2I alignment scores, and reference-image conditioning.

12. Common Mistakes

Calling CLIP generative; treating closed-set softmax as calibrated certainty; incorrect image preprocessing; comparing embeddings from different checkpoints; using random near-duplicate splits; relying on CLIP score alone for image quality.

13. Edge Cases / Limitations

CLIP can miss counting, spatial relations, OCR, fine-grained attributes, rare domains, and negation. Web supervision embeds social biases. In-batch negatives may include valid alternative captions. Cosine closeness is semantic association, not factual proof.

14. Variations

Variation	Change	Use	Importance
OpenCLIP	Open training/data/checkpoints	Reproducible alternatives	Practical
SigLIP	Sigmoid pairwise loss	Avoid global softmax dependence	Research
Domain CLIP	Fine-tuned domain encoders	Medical/products	Project
Multilingual CLIP	Multilingual text space	Cross-language search	Important
Cross-encoder VLM	Joint token interaction	Fine reasoning/reranking	Contrast topic

15. Related Topics

CLIP vs classifier: open-vocabulary prompt comparison versus fixed trained head. **CLIP vs BLIP:** dual-encoder alignment versus captioning/generative multimodal architectures. **CLIP vs cross-encoder:** efficient indexing

versus richer pair interaction. **CLIP score vs FID**: semantic alignment versus distributional image quality.

16. Interview Questions

1. **What does CLIP learn?** A shared embedding space for matching images and text.
2. **Why symmetric loss?** It trains both image→text and text→image retrieval.
3. **What are in-batch negatives?** All nonmatching examples in the current batch.
4. **Why normalize embeddings?** Similarity then depends on direction/semantics rather than vector magnitude.
5. **What does temperature do?** Scales logits and controls contrastive softmax sharpness.
6. **How does zero-shot classification work?** Compare an image embedding with prompt embeddings for candidate classes.
7. **Why prompt ensemble?** Different templates capture phrasing sensitivity; averaging improves robustness.
8. **Why is CLIP efficient for retrieval?** Each side is encoded independently and vectors can be indexed offline.
9. **What are false negatives?** Non-diagonal pairs that are semantically valid matches but are penalized.
10. **Can CLIP evaluate image realism?** Not reliably; it mainly measures semantic alignment.
11. **Why can softmax confidence mislead?** It is normalized over only the provided labels even if none is correct.
12. **How adapt CLIP to a domain?** Fine-tune/project with domain pairs, careful negatives, and held-out retrieval/calibration tests.

17. Practice Tasks

- **Small coding task:** build a zero-shot image classifier.
- **Dataset project:** create a text-to-image search index.
- **Experiment:** compare prompt templates and ensembles.
- **Debugging:** identify false negatives and preprocessing errors.
- **Extension:** fine-tune on a small domain pair dataset.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
Visual search	Text queries retrieve images	CLIP, FAISS, FastAPI	Unsplash subset	Retrieval deployment
Dataset auditor	Finds mismatched captions/duplicates	CLIP, Python	Conceptual Captions subset	Data quality
T2I evaluator	Scores alignment plus other metrics	CLIP, Diffusers	Prompt suite	Generative evaluation

19. Quick Revision

- **Key:** contrastively aligned dual encoders.
- **Formula:** symmetric InfoNCE over similarity matrix.
- **Metrics:** Recall@K, median rank, zero-shot accuracy, subgroup calibration.

- **Trap:** relative similarity is not calibrated truth or image quality.
- **One-liner:** "CLIP turns open-vocabulary vision into embedding retrieval."

20. Final Cheat Sheet

Item	Summary
Definition	Contrastively aligned image-text dual encoder
Input/output	Image or text → normalized shared embedding
Main steps	Encode → project → normalize → similarity → contrastive loss
Key hyperparameters	Encoders, projection size, temperature, batch/negatives
Metrics	Recall@K, median rank, zero-shot accuracy, calibration
Pros	Efficient zero-shot retrieval and reusable embeddings
Cons	Weak fine relations, false negatives, and learned bias
Best uses	Search, classification, filtering, T2I alignment

Multimodal Generation

1. Overview

Multimodal generation consumes and/or produces multiple modalities—text, images, audio, video, sensor data, or actions. Examples include answering questions about images, generating speech from text, producing captions, editing images through dialogue, and interleaving text with visuals. Systems may connect specialized encoders/decoders through a language model or tokenize all modalities into a unified sequence.

2. Intuition

Each modality is a different language. Encoders act as interpreters into a common representation; a reasoning backbone combines the evidence; modality-specific decoders translate its decisions back into words, pixels, or sound.

3. Prerequisites

Transformers and attention; CLIP/contrastive learning; image/audio tokenization; autoregressive and diffusion generation; instruction tuning; modality-specific preprocessing and evaluation.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Modality encoder	Converts raw input to tokens/features	Bridges different dimensions	ViT for image	Frozen vs trained

Concept	Meaning	Why	Example	Interview angle
Projector/adapter	Maps features to backbone width	Cheap alignment	MLP/Q-Former	Information bottleneck
Fusion	Combines modalities	Enables grounding/reasoning	cross-attention	Early/late fusion
Modality tokenizer	Discretizes image/audio/video	Unified autoregression	VQ codes	Quantization loss
Generative decoder	Produces target modality	Converts representation to output	LM or diffusion decoder	Separate heads vs unified tokens
Alignment tuning	Image-text/instruction training	Teaches correspondence and behavior	VQA pairs	Hallucination
Interleaving	Mixed modality sequence	Rich documents/dialogue	text-image-text	Ordering and context cost

5. Algorithm / Working Process

Preprocess each modality with its encoder/tokenizer; project features into a shared backbone; fuse using concatenation or cross-attention; train on paired and interleaved objectives; instruction-tune on conversational tasks. At inference, route inputs through encoders, reason/generate tokens, then send output tokens or conditioning to the appropriate text, image, or audio decoder.

6. Mathematical Foundation

Autoregressive unified modeling uses $p(y \mid x^{text}, x^{image}, \dots) = \prod_t p(y_t \mid y_{<t}, X)$. Cross-attention is $\text{softmax}(Q_{decoder} K_{modality}^T / \sqrt{d}) V_{modality}$. Training often combines

$$\mathcal{L} = \lambda_{LM} \mathcal{L}_{LM} + \lambda_{align} \mathcal{L}_{contrastive} + \lambda_{gen} \mathcal{L}_{image/audio}.$$

Loss weights matter because token counts and scales differ across modalities.

7. Practical Implementation

```
from PIL import Image
from transformers import pipeline

# A compact multimodal generation example: image -> generated caption.
captioner = pipeline("image-to-text", model="Salesforce/blip-image-captioning-base")
image = Image.open("scene.jpg").convert("RGB")
result = captioner(image, max_new_tokens=40)
print(result[0]["generated_text"])
```

8. Code Explanation

The pipeline performs checkpoint-specific vision preprocessing, encodes the image, and autoregressively generates text. Captioning is one direction of multimodal generation. A conversational vision-language model additionally formats image tokens with user instructions and should be evaluated for grounded answers, not just fluent captions.

9. Training / Evaluation

Curate paired, temporally aligned, licensed data; split by source/identity. Evaluate per task: CIDEr/SPICE for captioning, VQA accuracy, retrieval Recall@K, OCR, grounded localization, speech intelligibility, image/audio quality, hallucination, safety, latency, and human judgment. Use counterfactual tests where visual evidence changes but question wording stays fixed.

10. Complexity and Cost

Vision/video/audio create many tokens; self-attention may be quadratic in total sequence length. Multiple encoders and decoders increase memory and serving complexity. Feature resampling, pooling, patch reduction, caching, and modality routing lower cost but can discard detail.

11. Common Use Cases

Visual assistants, document understanding, captioning, accessibility, multimodal search, voice assistants, media creation, robotics interfaces, and medical decision support research.

12. Common Mistakes

Leaking answers through captions; evaluating language priors instead of visual grounding; wrong chat template/image preprocessing; mixing unrelated pairs; treating all modalities with one metric; ignoring modality-missing behavior and accessibility/safety.

13. Edge Cases / Limitations

Models may hallucinate objects, ignore images, fail on small text/spatial relations, synchronize audio/video poorly, or exploit dataset shortcuts. Long media exceeds context/compute. Confidence is often poorly calibrated; high-stakes outputs require external verification.

14. Variations

Variation	Change	Use	Importance
Dual encoder	Independent aligned features	Retrieval	Foundational
Encoder-adapter-LLM	Frozen vision/audio + projector	Efficient VLM	Essential
Cross-attention VLM	Deep modality interaction	Grounded generation	Important
Unified token model	One sequence of multimodal tokens	Interleaved generation	Research
Any-to-any system	Multiple input/output decoders	General assistant	Frontier

15. Related Topics

CLIP vs VLM: representation alignment versus conditional generation/reasoning. **Early vs late fusion:** token-level interaction versus combining decisions. **RAG vs multimodal context:** retrieved media becomes grounded input. **Unified vs modular:** simpler interface versus specialized quality and controllability.

16. Interview Questions

1. **What makes a model multimodal?** It processes or generates more than one data modality with learned cross-modal relationships.
2. **Why use a projector?** To map encoder features into the backbone's token dimension/distribution.
3. **What is modality fusion?** Combining information through concatenation, cross-attention, shared tokens, or late score fusion.
4. **How detect image ignoring?** Counterfactual images, image removal, contradiction tests, grounding benchmarks, and attention-independent attribution.
5. **Why freeze encoders?** Lower compute/data needs and retain pretrained representations; joint training may align better.
6. **What is a Q-Former-style adapter?** Learned queries compress and extract relevant features from a frozen modality encoder.
7. **Why tokenize images/audio?** Discrete tokens permit a unified autoregressive objective, at the cost of quantization error and long sequences.
8. **How evaluate hallucination?** Compare generated claims with annotated visible/audio evidence and object-level precision/recall.
9. **What is temporal alignment?** Correct synchronization of events across video, audio, subtitles, or actions.
10. **Why are multimodal systems costly?** Large raw inputs yield many tokens and multiple backbones/decoders.
11. **Modular vs unified model?** Modular systems use specialist components; unified systems share a sequence/model and may transfer better.
12. **What safety issue is unique?** Hidden instructions or harmful content can arrive through images/audio, not only text.

17. Practice Tasks

- **Small coding task:** run image captioning inference.
- **Dataset project:** benchmark VQA on counterfactual image pairs.
- **Experiment:** test OCR and small-object sensitivity.
- **Debugging:** repair image preprocessing/chat-template mismatch.
- **Extension:** add grounded speech output.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
Accessible scene assistant	Image → grounded spoken description	VLM, TTS, FastAPI	VizWiz	Social/product impact
Document analyst	Chart/image QA with citations	VLM, OCR, RAG	DocVQA	Multimodal grounding

Project	Function	Stack	Dataset	Resume value
Media search	Text↔image/audio retrieval	CLIP-like encoders, FAISS	Public media subset	Cross-modal systems

19. Quick Revision

- **Key:** encode, align/fuse, reason, and decode across modalities.
- **Formula:** conditional sequence loss plus optional alignment/generative losses.
- **Metrics:** task-specific quality, grounding, hallucination, latency, safety.
- **Trap:** fluency can hide ignored sensory evidence.
- **One-liner:** "Multimodal generation connects modality encoders and generators through aligned tokens or cross-attention."

20. Final Cheat Sheet

Item	Summary
Definition	Generation that consumes or produces multiple modalities
Input/output	Any supported media → text/image/audio/video
Main steps	Encode → project/fuse → reason/generate → decode
Key hyperparameters	Token budget, projector, loss weights, resolution
Metrics	Task score, grounding, hallucination, safety, latency
Pros	Rich, grounded, accessible interaction across media
Cons	High cost and cross-modal hallucination/alignment errors
Best uses	Assistants, documents, search, media and accessibility

Flow Matching

1. Overview

Flow matching trains a neural velocity field that transports samples from a simple base distribution to the data distribution through an ordinary differential equation (ODE). Unlike diffusion’s iterative stochastic reverse-process framing, it directly regresses a chosen conditional probability path’s velocity. Rectified flows use nearly straight paths and can enable efficient few-step generation.

2. Intuition

Think of particles starting as a cloud of random noise and moving through a time-varying wind field until they form images. Training shows the model points along many known bridges between noise and data; inference follows the learned wind with an ODE solver.

3. Prerequisites

Probability distributions, interpolation and conditional expectation, ODEs/Euler integration, neural vector fields, diffusion/score matching, and PyTorch autograd.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Probability path	p_t joins base and data	Defines transport problem	Gaussian bridge	Path choice
Velocity field	$v_t(x)$ moves probability mass	Generates via ODE	learned "wind"	Continuity equation
Conditional path	Tractable bridge for paired (x_0, x_1)	Gives supervised targets	linear interpolation	Marginalization theorem
Rectified flow	Straight interpolation target	Simple/few-step path	$x_t = (1 - t)x_0 + tx_1$	Reflow
ODE solver	Integrates $dx/dt = v_\theta(x, t)$	Sampling accuracy/cost	Euler/Heun	NFE trade-off
Conditioning	Adds text/class/control	Guided generation	$v(x, t, c)$	CFG possible

5. Algorithm / Working Process

Sample data x_1 , base noise x_0 , and $t \sim U[0, 1]$. Construct path sample x_t and analytic target velocity $u_t(x_t \mid x_0, x_1)$. Regress $v_\theta(x_t, t, c)$ to that velocity. For inference, sample $x(0)$ from the base and numerically integrate the learned ODE to $t = 1$.

6. Mathematical Foundation

The continuity equation is $\partial_t p_t(x) + \nabla \cdot (p_t(x)v_t(x)) = 0$. Conditional flow matching minimizes

$$\mathcal{L}_{CFM} = E_{t,x_0,x_1} \|v_\theta(x_t, t) - u_t(x_t \mid x_0, x_1)\|_2^2.$$

For rectified flow, $x_t = (1 - t)x_0 + tx_1$ and target velocity is $u_t = x_1 - x_0$. Euler sampling is $x_{k+1} = x_k + \Delta t v_\theta(x_k, t_k)$.

7. Practical Implementation

```
import torch
from torch import nn

net = nn.Sequential(nn.Linear(3, 128), nn.SiLU(), nn.Linear(128, 128), nn.SiLU(),
nn.Linear(128, 2))
x1 = torch.randn(256, 2) * 0.2 + torch.tensor([2.0, 2.0]) # toy data
x0 = torch.randn_like(x1)
t = torch.rand(len(x1), 1)
xt = (1 - t) * x0 + t * x1
```

```
target_velocity = x1 - x0
loss = nn.functional.mse_loss(net(torch.cat([xt, t], 1)), target_velocity)
loss.backward()

# Euler inference after training:
x = torch.randn(64, 2)
for k in range(50):
    tk = torch.full((len(x), 1), k / 50)
    x = x + net(torch.cat([x, tk], 1)) / 50
```

8. Code Explanation

The model receives current location and time and predicts a 2D velocity. Linear interpolation gives an exact per-pair target. The inference loop is explicit Euler; real systems train image-latent Transformers and use higher-order solvers. The toy data is illustrative, not a converged model after one backward pass.

9. Training / Evaluation

Track velocity MSE by time, sample FID/KID, precision/recall, solver NFE, truncation error, and likelihood when divergence integration is feasible. Compare samplers at equal NFE. Tune path/coupling, time sampling, target parameterization, solver, step schedule, CFG, EMA, and model scale.

10. Complexity and Cost

Training resembles one diffusion-model network evaluation per example. Inference cost is number of ODE function evaluations; straighter learned paths tolerate fewer steps. Exact likelihood needs divergence/Jacobian-trace estimation and extra compute. Solver memory can be constant for inference but adjoint training has numerical trade-offs.

11. Common Use Cases

Image/audio/video generation, fast latent generative models, molecular structure generation, simulation, and distribution transport.

12. Common Mistakes

Confusing flow matching with normalizing-flow coupling layers; reversing time endpoints; using the wrong target velocity for the chosen path; comparing methods at different NFE; unstable large Euler steps; forgetting conditioning in unconditional dropout.

13. Edge Cases / Limitations

Curved or crossing conditional paths can be hard to regress because conditional velocities average. Coarse solvers introduce artifacts. Few-step quality depends on path straightness and training. Theory and implementations vary in endpoint, schedule, and parameterization conventions.

14. Variations

Variation	Change	Use	Importance
Rectified flow	Linear paths	Simple/few-step generation	Essential modern
OT flow matching	Optimal-transport coupling	Straighter paths	Research important
Stochastic interpolants	General stochastic paths	Unifies model families	Advanced
Reflow	Retrain on model-induced pairs	Straighten trajectories	Research
Conditional flow	Adds semantic condition	T2I/multimodal	Project relevant

15. Related Topics

Flow matching vs diffusion: ODE velocity regression versus denoising/score objectives. **Flow matching vs normalizing flows:** continuous learned transport need not use architecturally invertible layers. **ODE vs SDE:** deterministic trajectory versus stochastic dynamics. **Velocity vs score:** related through the selected probability path but different prediction targets.

16. Interview Questions

1. **What does flow matching learn?** A time-dependent vector field transporting base density to data density.
2. **What is sampled at inference?** Base noise, then an ODE trajectory to the data endpoint.
3. **Why conditional flow matching?** Conditional paths have tractable samples/velocities even when marginal velocity is unknown.
4. **What is rectified flow?** Flow matching with straight interpolation and target $x_1 - x_0$.
5. **Why can straight paths be fast?** Low-curvature trajectories need fewer numerical steps.
6. **What is NFE?** Number of neural-network/vector-field evaluations made by the solver.
7. **Is sampling stochastic?** The ODE is deterministic given initial noise and condition; randomness comes from the initial sample.
8. **How does Euler error behave?** Global error is first order in step size under standard smoothness assumptions.
9. **What is the continuity equation?** Conservation law linking density evolution to velocity flow.
10. **Can flow models estimate likelihood?** Yes via change-of-variables/divergence integration, often at extra cost.
11. **What is coupling?** How base and data samples are paired to define conditional paths.
12. **Main implementation trap?** Mixing time/path/velocity conventions from different formulations.

17. Practice Tasks

- **Small coding task:** train a 2D rectified flow.
- **Dataset project:** generate MNIST using latent flow matching.
- **Experiment:** compare Euler and Heun at equal NFE.
- **Debugging:** identify reversed endpoints or wrong velocity targets.
- **Extension:** add class conditioning and guidance.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
2D flow lab	Interactive transport visualization	PyTorch, Plotly	Synthetic mixtures	Strong intuition
FM image model	Generates small images	PyTorch	MNIST/CIFAR-10	Modern research skill
Solver benchmark	Quality vs NFE	torchdiffeq/PyTorch	Saved checkpoint	Numerical systems

19. Quick Revision

- **Key:** regress a transport velocity, then integrate an ODE.
- **Formula:** $E\|v_\theta(x_t, t) - u_t\|^2$.
- **Metrics:** sample quality, coverage, NFE, latency.
- **Trap:** target must match path convention.
- **One-liner:** "Flow matching turns generation into learned probability transport."

20. Final Cheat Sheet

Item	Summary
Definition	Learned ODE velocity transporting base density to data
Input/output	Base noise + condition → ODE-transported sample
Main steps	Sample endpoints/time → regress velocity → integrate
Key hyperparameters	Path, coupling, time sampling, solver, steps, CFG
Metrics	FID/KID, coverage, NFE, solver error, latency
Pros	Simple regression objective and few-step potential
Cons	Solver, path, coupling, and convention sensitivity
Best uses	Modern image/audio/video and scientific generation

Video Generation

1. Overview

Video generation produces temporally coherent frame sequences from text, images, video, actions, or motion controls. Modern systems extend latent diffusion/flow Transformers with time-aware attention and compressed video autoencoders. Applications include storyboards, animation, simulation, advertising, virtual production, training data, and interactive environments.

2. Intuition

Generating a video is not generating many independent pictures: objects must keep identity, move plausibly, obey the camera, and remain consistent when occluded. The model must learn appearance and dynamics jointly.

3. Prerequisites

Image/video tensors and codecs; optical flow and temporal consistency; diffusion/flow matching; spatial-temporal attention; 3D convolutions; distributed GPU training and video metrics.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Video latent	Compressed space-time tensor	Reduces huge cost	VAE tokens	Temporal compression
Temporal attention	Connects tokens across frames	Maintains identity/motion	track a car	Full vs factorized
Motion condition	Specifies dynamics	User control	camera pan/pose	Sparse controls
Image-to-video	First/reference frame condition	Strong appearance anchor	animate photo	Drift
Temporal coherence	Stable objects and motion	Main quality requirement	no flicker	Metrics imperfect
Causal generation	Predicts future without future context	Streaming/world modeling	next frames	Error accumulation
Shot consistency	Coherence over long duration	Storytelling	character continuity	Hierarchical generation

5. Algorithm / Working Process

Decode and sample clips; spatially/temporally compress them; add noise to video latents; condition a 3D U-Net or video Transformer on text/reference/action; predict noise/velocity; decode denoised latents. Inference samples a complete clip or extends chunks with overlap, keyframes, and memory. Postprocessing may interpolate frames or upscale.

6. Mathematical Foundation

For latent video $z \in \mathbb{R}^{T \times H \times W \times C}$, diffusion loss remains $E\|\epsilon - \epsilon_\theta(z_t, t, c)\|^2$. Factorized attention applies spatial attention within frames and temporal attention across the same spatial locations, reducing full attention over $T H W$ tokens. Temporal consistency can include $\sum_t \|\hat{I}_{t+1} - W(\text{hat} I_t, F_t)\|$ using flow warp W .

7. Practical Implementation

```
import torch
from diffusers import DiffusionPipeline
from diffusers.utils import export_to_video

device = "cuda" if torch.cuda.is_available() else "cpu"
pipe = DiffusionPipeline.from_pretrained(
    "cerspense/zeroscope_v2_576w",
    torch_dtype=torch.float16 if device == "cuda" else torch.float32,
).to(device)
frames = pipe("a toy robot walking across a desk, locked camera",
              num_inference_steps=30, num_frames=24).frames[0]
export_to_video(frames, "generated_video.mp4", fps=8)
```

8. Code Explanation

The pipeline generates a batch of frames jointly, then encodes them into a playable video. Exact arguments and hardware support are checkpoint/version specific. Production must set seeds, dimensions, duration, fps, codec, memory controls, and content policies; generating more frames is not equivalent to higher fps.

9. Training / Evaluation

Split by source video/scene, remove duplicate clips and caption leakage, preserve fps metadata. Evaluate FVD, frame FID, CLIP alignment, optical-flow consistency, subject/face identity, action correctness, physics, motion magnitude, human preference, and latency. FVD needs many samples and is preprocessing-sensitive. Test static, fast motion, occlusion, camera movement, and long horizons.

10. Complexity and Cost

Tokens scale with frames $\times$ spatial positions. Full attention is quadratic in that product; factorized/windowed attention reduces cost. Video decoding, storage, and data loading are also bottlenecks. Long clips require multi-GPU training, checkpointing, mixed precision, temporal compression, and chunking.

11. Common Use Cases

Previsualization, animation, ad concepts, image animation, video editing, simulation data, training environments, education, and short social content.

12. Common Mistakes

Training on random frames rather than clips; leaking neighboring clips across splits; evaluating frames only; confusing duration/fps/frame count; ignoring aspect ratio; generating chunks without overlap consistency; claiming physical correctness from visual plausibility.

13. Edge Cases / Limitations

Long-horizon identity, object permanence, readable text, hands, fast motion, collisions, and causal physics remain difficult. Flicker and morphing occur. Training data has consent/copyright risks; generated video increases deepfake risk and serving cost.

14. Variations

Variation	Change	Use	Importance
Text-to-video	Text only	Open creation	Essential
Image-to-video	Reference frame	Animate images	Essential
Video-to-video	Source motion/content	Editing/style	Project
Autoregressive video	Next tokens/frames	Long/causal prediction	Research
Diffusion/flow video	Joint denoising/transport	High-quality clips	Modern essential
Hierarchical video	Plans keyframes then fills	Long duration	Frontier

15. Related Topics

Video vs image generation: adds temporal dynamics and identity. **Optical flow vs flow matching:** pixel-motion estimation versus probability transport. **Video generation vs world model:** visual synthesis versus action-conditioned predictive environment model. **3D-aware video:** explicit/implicit geometry improves view consistency.

16. Interview Questions

1. **Why not generate frames independently?** Identity, motion, lighting, and geometry would flicker without temporal dependence.
2. **What is temporal attention?** Attention linking features across frames to model motion and consistency.
3. **What is FVD?** Distribution distance between real/generated video features, analogous to FID but spatiotemporal.
4. **Why is video expensive?** Many more tokens, temporal activations, storage bandwidth, and iterative sampling.
5. **Text-to-video vs image-to-video?** The latter anchors appearance/composition with a reference frame.
6. **What is temporal compression?** Video autoencoder reduces frames/spatial size before generation.
7. **How evaluate motion?** Flow consistency, action recognition, tracking/identity, motion statistics, and human review.
8. **What causes long-video drift?** Autoregressive/chunk errors accumulate and context cannot retain all prior state.
9. **How extend duration?** Overlapping windows, memory tokens, keyframe hierarchy, retrieval, or causal latent models.
10. **Does high frame quality imply good video?** No; independently excellent frames can flicker or violate dynamics.
11. **What is camera conditioning?** Explicit trajectories/motion labels guide viewpoint movement.
12. **Major safety risk?** Convincing identity impersonation and misleading synthetic events.

17. Practice Tasks

- **Small coding task:** generate and export prompt-conditioned clips.
- **Dataset project:** score motion on an action-video subset.

- **Experiment:** vary fps, frame count, and temporal guidance.
- **Debugging:** detect neighboring-clip split leakage and flicker.
- **Extension:** add first-frame image conditioning.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
Motion benchmark	Scores consistency/latency	Diffusers, OpenCV	Prompt suite	Evaluation systems
Product animator	Still image → short turntable	Video diffusion, Gradio	Licensed products	Commercial workflow
Action simulator	Generates action-conditioned clips	PyTorch	BAIR robot pushing	World-model bridge

19. Quick Revision

- **Key:** jointly model appearance and temporal dynamics.
- **Formula:** video-latent denoising/velocity loss.
- **Metrics:** FVD, alignment, identity, flow consistency, preference.
- **Trap:** per-frame quality ignores temporal failure.
- **One-liner:** “Video generation adds temporal tokens and dynamics constraints to visual generation.”

20. Final Cheat Sheet

Item	Summary
Definition	Generative modeling of coherent visual sequences
Input/output	Text/image/video/action → frame sequence
Main steps	Encode clip/condition → generate latent video → decode/export
Key hyperparameters	Frames, fps, resolution, steps, CFG, temporal window
Metrics	FVD, alignment, identity, flow consistency, preference
Pros	Dynamic content, animation, and simulation data
Cons	Extreme cost, long-horizon drift, and deepfake risk
Best uses	Short clips, animation, editing, simulation

3D Generation

1. Overview

3D generation creates geometry and appearance as meshes, point clouds, voxels, neural radiance fields (NeRFs), Gaussian splats, or implicit fields from text, images, or scans. It supports games, AR/VR, product

design, robotics simulation, digital twins, and content creation. Multi-view consistency and usable geometry matter beyond attractive rendered views.

2. Intuition

An image generator paints one viewpoint; a 3D generator must build an object that still makes sense when walked around. A differentiable renderer is the camera through which 2D supervision can teach a 3D representation.

3. Prerequisites

3D coordinates/transforms/cameras; rendering and ray sampling; meshes/point clouds/implicit fields; CNNs/Transformers/diffusion; optimization and differentiable rendering.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Representation	Data structure for shape/appearance	Determines quality/editability	mesh, NeRF, splat	Explicit vs implicit
Camera model	Maps 3D to image rays	Required for views	intrinsics/extrinsics	Coordinate conventions
Differentiable rendering	Gradients from pixels to 3D	Enables image supervision	volume rendering	Occlusion
Multi-view consistency	Same object across cameras	Defines genuine 3D	stable backside	Janus problem
Geometry prior	Encourages plausible surfaces	2D supervision is ambiguous	smooth normals	Regularization
Texture/material	Surface appearance	Asset realism	UV map/PBR	Baked lighting

5. Algorithm / Working Process

Choose representation. For multi-view reconstruction, encode observations, predict/optimize 3D parameters, render training cameras, and minimize photometric/perceptual/geometry losses. For text-to-3D, initialize a 3D scene, render random views, use a pretrained T2I prior (score distillation) or a native 3D generator, regularize geometry, then extract/clean mesh and texture for export.

6. Mathematical Foundation

NeRF represents density σ and color c along ray $r(t) = o + td$. Rendered color is

$$C(r) = \int T(t)\sigma(r(t))c(r(t), d)dt, \quad T(t) = \exp\left(-\int_{t_n}^t \sigma(r(s))ds\right).$$

Discrete alpha compositing approximates this integral. Reconstruction uses $\sum_r \|C(r) - C_{gt}(r)\|^2$. Meshes often add Laplacian/normal/eikonal regularization. Score distillation uses gradients from a frozen 2D diffusion

prior on rendered noisy views.

7. Practical Implementation

```
import torch

def volume_render(rgb, sigma, delta):
    """rgb:[R,S,3], sigma/delta:[R,S] -> ray colors."""
    alpha = 1 - torch.exp(-sigma * delta)
    trans = torch.cumprod(torch.cat([torch.ones_like(alpha[:, :1]),
                                     1 - alpha + 1e-10], dim=1), dim=1)[:, :-1]

    weights = alpha * trans
    return (weights.unsqueeze(-1) * rgb).sum(dim=1), weights

rgb = torch.rand(8, 64, 3)
sigma = torch.rand(8, 64) * 5
delta = torch.full((8, 64), 0.02)
colors, weights = volume_render(rgb, sigma, delta)
assert colors.shape == (8, 3)
```

8. Code Explanation

Density and interval length produce opacity `alpha`. Cumulative transmittance is the probability a ray reaches a sample without terminating. Their product gives each sample's contribution. A full NeRF network predicts `rgb, sigma` from positions/directions and optimizes rendered colors against calibrated images.

9. Training / Evaluation

Split by object/scene, not views from the same object. Evaluate novel-view PSNR/SSIM/LPIPS, geometry Chamfer distance/F-score/normal consistency, CLIP alignment, multi-view consistency, mesh validity, polygon count, render speed, and human preference. Validate camera calibration and background masks. Inspect assets in a real renderer, not only training viewpoints.

10. Complexity and Cost

Naive NeRF evaluates many samples per ray and trains slowly; grids/hash encodings accelerate it. Meshes render fast but topology generation is difficult. 3D data is scarce and expensive. Text-to-3D optimization may require many diffusion-guided renders; native feed-forward models are faster but data-hungry.

11. Common Use Cases

Game/film assets, product digitization, AR/VR, robotics simulation, architectural visualization, digital humans, and synthetic 3D datasets.

12. Common Mistakes

Leaking views of one object across splits; wrong camera units/axes; evaluating renders only; ignoring mesh topology/UVs; confusing NeRF with an editable mesh; overfitting known views; reporting CLIP score without geometry checks.

13. Edge Cases / Limitations

Transparent/reflective surfaces, thin structures, unseen backsides, topology, baked lighting, and exact scale are hard. 2D priors can create Janus/multi-face artifacts. Generated assets often require retopology, UV cleanup, rigging, and material correction.

14. Variations

Variation	Change	Use	Importance
Mesh	Vertices/faces	Editable/exportable assets	Essential
Point cloud	Unordered points	Scans/geometry generation	Basic
NeRF	Neural density/radiance field	Novel-view synthesis	Essential
Gaussian splatting	Explicit anisotropic Gaussians	Fast scene rendering	Modern important
SDF/occupancy	Implicit surface	Clean geometry	Research
3D diffusion	Generates native 3D latent/shape	Scalable asset synthesis	Frontier

15. Related Topics

NeRF vs mesh: view synthesis quality versus editability/render portability. **NeRF vs Gaussian splats:** implicit neural field versus explicit fast primitives. **Reconstruction vs generation:** recovering one observed scene versus sampling new assets. **2D-guided vs native 3D:** abundant image priors versus stronger 3D consistency.

16. Interview Questions

1. **Why is 3D generation harder than images?** It must remain consistent across viewpoints and produce valid geometry/materials.
2. **What does a NeRF learn?** Density and view-dependent radiance as functions of 3D position and direction.
3. **What is transmittance?** Probability light travels to a ray sample without earlier absorption.
4. **Why need camera poses?** They define rays linking pixels to 3D coordinates.
5. **What is the Janus problem?** Multiple front-like faces appear because independent 2D guidance lacks global 3D consistency.
6. **How evaluate geometry?** Chamfer distance, F-score, normal consistency, watertightness, topology, and visual inspection.
7. **Why is CLIP score insufficient?** A few renders can match text while hidden views/geometry are invalid.
8. **NeRF vs Gaussian splatting?** NeRF queries a neural field; splatting renders optimized explicit Gaussians rapidly.
9. **What is differentiable rendering?** A renderer whose output gradients update geometry/material/camera parameters.
10. **Why split by object?** Views from the same object in train/test cause severe leakage.

11. **What is score distillation?** Use a frozen image diffusion model’s denoising signal to optimize rendered 3D parameters.
12. **What makes an asset production-ready?** Valid scale, topology, UV/materials, low artifacts, manageable polygons, and often rigging.

17. Practice Tasks

- **Small coding task:** implement alpha compositing.
- **Dataset project:** train a tiny NeRF from multi-view images.
- **Experiment:** compare mesh, NeRF, and view-space metrics.
- **Debugging:** fix camera axes, scale, or pose errors.
- **Extension:** export and inspect a generated mesh/material asset.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
NeRF capture	Phone images → novel views	PyTorch/nerfstudio	Own static object	3D pipeline
Asset QA	Scores mesh/render quality	trimesh, Blender	Objaverse subset	Production tooling
Text-to-3D benchmark	Audits multi-view consistency	Diffusion, CLIP	Prompt suite	Research evaluation

19. Quick Revision

- **Key:** generate representation valid across views, not one image.
- **Formula:** volume rendering with transmittance-weighted color.
- **Metrics:** novel-view quality + geometry/asset validity.
- **Trap:** attractive renders can hide broken 3D.
- **One-liner:** “3D generation must model geometry, appearance, and cameras under multi-view consistency.”

20. Final Cheat Sheet

Item	Summary
Definition	Generation of geometry and appearance valid across viewpoints
Input/output	Text/images/noise → mesh, field, points, or splats
Main steps	Represent → render → compare/guidance → regularize → export
Key hyperparameters	Representation size, rays/samples, views, guidance, geometry weights
Metrics	PSNR/SSIM/LPIPS, Chamfer/F-score, normals, asset validity
Pros	Viewable, renderable, and potentially editable assets
Cons	Scarce data, geometry ambiguity, and cleanup requirements

Item	Summary
Best uses	AR/VR, games, products, simulation, reconstruction

World Models

1. Overview

A world model learns how an environment evolves, often predicting future latent states or observations conditioned on actions. It can support planning, model-based reinforcement learning, robotics, autonomous systems, games, and embodied agents. A useful world model represents state, dynamics, uncertainty, rewards, and terminal events—not merely visually convincing video.

2. Intuition

Humans mentally simulate: “If I push this cup, it may fall.” A world model is an agent’s internal simulator. The agent can imagine several action sequences, estimate outcomes, and execute the safest or highest-reward plan in the real environment.

3. Prerequisites

MDPs/POMDPs, states/actions/rewards; sequence models and latent variables; VAEs/diffusion/video prediction; reinforcement learning and planning; uncertainty, system identification, and offline-data bias.

4. Core Concepts

Concept	Meaning	Why	Example	Interview angle
Latent state s_t	Compact task-relevant history	Raw pixels are large/partially observed	scene + velocity	Markov sufficiency
Transition model	$p(s_{t+1} \mid s_t, a_t)$	Predicts action consequences	steering changes pose	Stochastic dynamics
Observation model	$p(o_t \mid s_t)$	Decodes predicted sensory data	latent $\rightarrow$ frame	Optional for latent planning
Reward/continuation model	Predicts value signals/termination	Enables imagined control	collision penalty	Model bias
Rollout	Repeated imagined transitions	Evaluates action sequence	trajectory	Compounding error
Planner/policy	Selects action using model	Converts prediction to decisions	MPC	Exploitation of model errors

Concept	Meaning	Why	Example	Interview angle
Uncertainty	Represents multiple futures/unknown regions	Safer planning/exploration	pedestrian motion	Epistemic vs aleatoric

5. Algorithm / Working Process

Collect trajectories (o_t, a_t, r_t, d_t) . Encode observations into latent states and update state using past state/action/new observation. Train transition, reward, continuation, and optional reconstruction predictors. During imagination, start from inferred current state, simulate candidate actions without the environment, score returns/uncertainty, then execute an action via MPC or train a policy/value function on imagined rollouts. Continually validate predicted and real outcomes.

6. Mathematical Foundation

An MDP factorizes $p(s_{t+1}, r_t | s_t, a_t)$. A latent model may optimize

$$\mathcal{L} = \lambda_o[-\log p(o_t | s_t)] + \lambda_r[-\log p(r_t | s_t)] + \lambda_d[-\log p(d_t | s_t)] + \beta D_{KL}(q(s_t) || p(s_t | s_{t-1}, a_{t-1})).$$

Planning seeks $a_{t:t+H-1}^* = \arg \max E[\sum_{k=0}^{H-1} \gamma^k r_{t+k} + \gamma^H V(s_{t+H})]$. Multi-step prediction error compounds; probabilistic transitions approximate several valid futures.

7. Practical Implementation

```
import torch
from torch import nn

class LatentDynamics(nn.Module):
    def __init__(self, state_dim=32, action_dim=4):
        super().__init__()
        self.transition = nn.Sequential(nn.Linear(state_dim + action_dim, 128),
nn.SiLU(), nn.Linear(128, state_dim))
        self.reward = nn.Sequential(nn.Linear(state_dim, 64), nn.SiLU(),
nn.Linear(64, 1))

    def forward(self, state, action):
        next_state = state + self.transition(torch.cat([state, action], -1))
        return next_state, self.reward(next_state)

model = LatentDynamics()
state = torch.randn(16, 32)
actions = torch.randn(16, 5, 4)
predicted_return = torch.zeros(16, 1)
for t in range(actions.size(1)):
    state, reward = model(state, actions[:, t])
    predicted_return += (0.99 ** t) * reward
assert predicted_return.shape == (16, 1)
```

8. Code Explanation

The residual transition predicts a latent state change conditioned on action; the reward head scores imagined states. The loop is a differentiable five-step rollout. A complete system also learns an observation encoder, stochastic state, continuation probability, and trains against real next-state/reward targets. Planning would compare many candidate action sequences or optimize them.

9. Training / Evaluation

Split by episode, environment seed, geography, or time. Evaluate one-step and multi-step latent/observation error, reward/termination accuracy, calibration, rollout consistency, downstream return, sample efficiency, safety violations, and generalization. Closed-loop environment performance matters most. Use action-balanced data, ensembles/uncertainty, scheduled rollout lengths, and regular re-grounding in observations.

10. Complexity and Cost

Training video-scale observation models is expensive, but latent imagination can be far cheaper than real interaction. Planning cost grows with horizon $\times$ candidates $\times$ model evaluations. Longer horizons worsen error. Policies distilled from planning offer fast inference; MPC replans online and is more robust but costly.

11. Common Use Cases

Robot manipulation/navigation, autonomous-driving simulation, game-playing agents, industrial control, data-efficient RL, counterfactual planning, and interactive generative environments.

12. Common Mistakes

Calling any video generator a world model; random transition leakage; optimizing pixel accuracy rather than control-relevant state; deterministic prediction of multimodal futures; evaluating only one-step loss; planning far beyond validated horizon; letting the planner exploit model errors.

13. Edge Cases / Limitations

Rare events, contact physics, partial observability, other agents, distribution shift, and long horizons are hard. A planner may find adversarial unrealistic trajectories that the model scores highly. Offline datasets omit counterfactual actions. Visually plausible futures may have wrong causal/reward dynamics, creating safety risks.

14. Variations

Variation	Change	Use	Importance
Pixel world model	Predicts observations directly	Interpretability/video	Basic
Latent world model	Predicts compact states	Efficient planning	Essential
Deterministic dynamics	One future	Nearly deterministic tasks	Basic
Stochastic dynamics	Distribution over futures	Uncertain environments	Essential
MPC model	Online candidate planning	Control constraints	Important

Variation	Change	Use	Importance
Imagination-trained actor	Policy/value learn in model	Fast deployment	Modern RL

15. Related Topics

World model vs video model: action/reward/causal usefulness versus visual prediction. **Model-based vs model-free RL:** planning through learned dynamics versus directly learned policy/value. **MPC vs policy:** online replanning versus amortized decisions. **State-space model vs Transformer:** recurrent compact belief versus attention over history; hybrids exist.

16. Interview Questions

1. **What is a world model?** A learned predictive model of environment state transitions, often including observations, rewards, and termination.
2. **Why use latent states?** They compress high-dimensional observations and can retain control-relevant information.
3. **What makes a state Markov?** It contains enough history that the next-state distribution depends only on current state/action.
4. **Why model stochasticity?** Many environments have genuinely multiple valid futures or hidden variables.
5. **What is imagination?** Rolling the learned transition model forward without real environment interaction.
6. **How does MPC use it?** Score candidate action sequences, execute the first action, observe, and replan.
7. **What is compounding error?** Small transition errors feed future predictions and grow across rollout steps.
8. **How mitigate model exploitation?** Ensembles/uncertainty penalties, short-horizon replanning, conservative objectives, constraints, and real validation.
9. **Why is pixel loss insufficient?** Control depends on objects, dynamics, rewards, and rare hazards, not average visual fidelity.
10. **World model vs simulator?** A simulator is usually engineered from known rules; a world model learns dynamics from data.
11. **How evaluate it?** Multi-step calibration and closed-loop task return/safety under held-out environments.
12. **Why split by episode?** Adjacent transitions are highly correlated; random rows leak trajectories into test data.

17. Practice Tasks

- **Small coding task:** learn CartPole latent dynamics.
- **Dataset project:** train on episode-level robot or driving trajectories.
- **Experiment:** compare one-step and multi-step predictive error.
- **Debugging:** detect episode leakage and planner exploitation.
- **Extension:** add random-shooting MPC and ensemble uncertainty penalties.

18. Project Ideas

Project	Function	Stack	Dataset	Resume value
CartPole MPC	Learns dynamics and plans actions	PyTorch, Gymnasium	Collected rollouts	Complete model-based RL
Robot latent model	Predicts action-conditioned video/state	PyTorch	BAIR robot pushing	Embodied AI research
Driving risk simulator	Forecasts trajectories/rewards	Transformers, CV	nuScenes subset	Safety-critical modeling

19. Quick Revision

- **Key:** predict action-conditioned futures for planning/control.
- **Formula:** $p(s_{t+1}, r_t \mid s_t, a_t)$ and finite-horizon return.
- **Metrics:** multi-step calibration, reward accuracy, return, safety.
- **Trap:** visually plausible is not causally correct.
- **One-liner:** "A world model earns its name when imagined rollouts support better real decisions."

20. Final Cheat Sheet

Item	Summary
Definition	Learned action-conditioned model of environment evolution
Input/output	State/history + action → future state/observation/reward/done
Main steps	Encode → transition → imagine → score → plan/policy → reobserve
Key hyperparameters	Latent size, horizon, discount, loss weights, uncertainty, candidates
Metrics	Multi-step calibration, reward accuracy, return, safety violations
Pros	Data-efficient planning and counterfactual evaluation
Cons	Model bias, exploitation, and compounding rollout errors
Best uses	Robotics, games, control, simulation, embodied agents